Digital Mohalla: A Trust-Based Platform for Neighbors

FYP-1 Mid-Evaluation Report (Iteration 1)

Project ID: F26-188

Project Team

Fatima Khursheed	23i-0746
Areeba Javed	23i-0570
Minahil Kashif	23i-0554

Session 2023–2027

Supervised by

Dr. Ali Zeeshan

FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *Digital Mohalla: A Trust-Based Platform for Neighbors* (project ID F26-188).

The report describes the work of Iteration 1: the Part A design baseline and the Part B implementation in three vertical slices (A: workflow skeleton, B: assurance core, C: human review loop), as it stands in the repository on 14 October 2026. Items that are deferred, provisional or not yet measured are stated as such where they appear.

Student 1: Fatima Khursheed (23i-0746) Signature: _____

Student 2: Areeba Javed (23i-0570) Signature: _____

Student 3: Minahil Kashif (23i-0554) Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Generative AI tools were used during the project: Claude Code for drafting code, tests and documents, and GitHub Copilot for an earlier report draft and automated pull-request review comments. Their use is declared per module in Section 8.7 and recorded entry by entry in the project's AI usage log (`docs/ai-usage-log.md`). Open-source libraries are listed with their versions and licences in Section 8.8.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Each member owns the modules assigned in the Iteration 1 plan (Table 3.3) and has reviewed, tested and can explain the design, code and tests of those modules. All data used in development and testing is synthetic and is labelled as synthetic.

Abstract

Neighbourhood lending ends in a physical handover between people who often have little or no shared history, so a platform has to decide whether a loan is safe from evidence that is sparse, indirect and open to manipulation. Digital Mohalla is a neighbourhood resource-sharing platform built around that decision. Its core is an assurance engine that combines a recency-weighted Beta posterior over a borrower's verified loan outcomes, bounded k-hop evidence through the community's trust graph and interpretable manipulation signals, and returns one of four evidence states with its uncertainty, recommended safeguards and a written explanation; uncertain or suspicious cases go to a human moderator instead of being rejected automatically.

Iteration 1 delivered a live core: an event-sourced borrowing lifecycle (14 states; every lifecycle rule, namely allowed transitions, actor roles, no double booking and append-only events, is also enforced in PostgreSQL), the assurance engine, disputes and moderator decisions, a resident and moderator app, and a CI/CD pipeline deploying to a public instance. 623 backend and 29 frontend tests pass. All seven evaluation scenarios required at mid-term return the expected state and explanation, and eleven of the thirteen scenarios reproduce their expected state end to end on synthetic data. Parameters remain hypotheses until the FYP-2 evaluation, which this report plans.

Acknowledgments

We thank our supervisor, Dr. Ali Zeeshan, for guidance throughout the project, and the proposal defence panel, whose feedback on location and on the depth of the technical challenge shaped the direction of this iteration. We also thank the faculty of the FAST School of Computing and the maintainers of the open-source tools listed in Section 8.8.

Executive Summary

Problem. Neighbours lend to one another, but a loan ends with an item handed to someone the lender may not know. Existing platforms either show one averaged rating, which hides how little evidence stands behind it, or show nothing at all. Neighbourhood evidence is sparse (most members have few loans), role-dependent (a good lender is not necessarily a good borrower), indirect (a borrower may be known to the lender's neighbours but not to the lender) and manipulable (a small group can rate itself up). Digital Mohalla treats the question *"is there enough trustworthy evidence for this loan?"* as its complex computing problem.

Response to the proposal defence. The panel approved the proposal with minor modifications and made three points: show that location is a real issue, the platform has positive social potential, and the technical challenge looked limited. The third point changed the project most. The team narrowed the first iteration to the assurance engine and the transaction lifecycle that feeds it, and answered the location point with a comparison of five locality methods, a threat model and eighteen test checks on the chosen rule (Chapter 3).

What Iteration 1 delivered. All three planned slices are complete and merged:

- **Slice A, workflow skeleton:** an append-only, hash-chained event store; a 14-state borrowing lifecycle whose transitions are checked in Python and again by a PostgreSQL foreign key; double booking prevented by a database exclusion constraint; login and a REST API; a resident app; CI and CD.
- **Slice B, assurance core:** outcome extraction from closed loans, a recency-weighted Beta posterior with a 90% credible interval and similarity weighting, trust edges and bounded k-hop search ($k = 3$), a decision policy of rules R1–R9 (R6 in two variants) with safeguards and plain-language explanations, a scenario suite in CI and the lender's evidence screen.
- **Slice C, human loop:** partner-clustering and review-similarity signals with a retaliation check, disputes that freeze a loan, moderator decisions on disputes and holds, privacy checks on every API response and log line, request fuzzing, and the moderator app.

Evidence. 623 backend tests (unit, integration on PostgreSQL 16, property-based and scenario tests) and 29 frontend tests pass in CI, with 97% line coverage of the backend code, and every pull request runs them before merge. All seven scenarios required at mid-term (ES-01 to ES-06 and ES-13) return the documented state *and* explanation, against a target of seven of thirteen (NFR-Q-01); eleven of the thirteen reproduce their expected state end to end on synthetic data. At $k = 3$ the indirect-evidence search takes 4.5 ms at the 95th percentile on a 2,026-user synthetic graph and finds a path for 93.2% of loan pairs. The API and the web app are deployed on Render with a Neon PostgreSQL database, from the `main` branch after CI passes.

Honest limits. All parameters (half-life, thresholds, decay, signal weights) are initial hypotheses checked only on synthetic data. The locality method is designed but not built; photo uploads, the audit-log screen, three manipulation signals (S2, S3, S5) and the scheduler move to Iteration 2. Generative AI (Claude Code) drafted most first-draft code under the team's direction; its use is declared per module in Section 8.7.

Next. Iteration 2 completes the core modules (membership and locality, the remaining lifecycle timers, the remaining signals and reviews, case information requests and appeals) and builds the simulator, baselines and FYP-2 evaluation plan that will test whether the engine's uncertainty actually improves decisions over simple averages.

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations xiv

1 Introduction and Project Vision 1

1.1 Project Overview 1

1.2 Problem Statement 1
1.3 Problem Context and Motivation 2
1.4 Project Vision 2
1.5 Project Objectives 2
1.6 Project Scope 3
1.7 Project Modules 3
1.8 Stakeholders and User Classes 5
1.9 Success Criteria 5
1.10 Constraints and Assumptions 6
1.11 Ethical, Legal and DEI Considerations 7

2 Research and Existing Solutions **8**
2.1 Research Method / Approach 8
2.2 Existing Solutions / Products 8
2.3 Comparison of Existing Solutions 9
2.4 Limitations and Gaps 10
2.5 Proposed Solution 10
2.6 Complex Computing Problem 10
2.7 Technical Challenges 10
2.8 Research Findings and Design Decisions 12

3 Proposal Feedback and Iteration Plan **15**
3.1 Proposal Feedback Summary 15
3.2 Proposal Action Register 15
3.3 Iteration Strategy 18
3.4 Iteration 1 Plan 18
3.5 Iteration 2 Plan 19
3.6 Work Breakdown and Responsibilities 20
3.7 Progress Against Plan 21
3.8 Delays, Deviations and Recovery Plan 22

4 Software Requirements **23**
4.1 User Interactions / Use Cases 23
4.2 Use Case Diagram 24
4.3 Detailed Use Cases 24
4.4 Functional Requirements 25
4.5 Non-Functional Requirements 27
4.6 Acceptance Criteria 30
4.7 Requirements Traceability 31

5 System Design **33**

5.1 System Architecture . . . 33
5.2 Architectural Components . . . 34
5.3 Interfaces and Dependencies . . . 37
5.4 Architecture Decisions and Rationale . . . 39
5.5 Design Evolution . . . 41
5.6 Domain Model . . . 43
5.7 Data Design . . . 43
5.8 Process / Behavior Design . . . 45

6 Iteration 1: Design and Implementation 52
6.1 Iteration Objectives . . . 52
6.2 Selected Use Cases / Features . . . 52
6.3 Design . . . 53
6.4 Implementation . . . 53
6.5 Completed Features / Module . . . 54
6.6 Technical Implementation Details . . . 55
6.7 Algorithms and Technical Decisions . . . 56
6.8 Tools, Frameworks and Technologies . . . 61
6.9 Demonstration Evidence . . . 61
6.10 Iteration Outcome . . . 62

7 Testing and Early Validation 63
7.1 Testing Strategy . . . 63
7.2 Test Levels and Techniques . . . 64
7.3 Test Cases . . . 64
7.4 Test Results . . . 66
7.5 Acceptance Testing / Validation . . . 69
7.6 Edge and Failure Cases . . . 69
7.7 Defects and Corrective Actions . . . 69
7.8 Current Validation Status . . . 71

8 Project Management and Development Evidence 72
8.1 Work Completed . . . 72
8.2 Work Remaining . . . 72
8.3 Individual Responsibilities . . . 73
8.4 Contribution of Each Team Member . . . 73
8.5 Repository and Version Control Evidence . . . 73
8.6 Milestones and Commit Evidence . . . 74
8.7 Tools and Tool-Assisted Development . . . 75
8.8 External Components and Dependencies . . . 78
8.9 Risks and Mitigation . . . 80

8.10 Security and Privacy Considerations . . . 81
8.11 Current Limitations . . . 82

9 Remaining FYP Plan **83**
9.1 Remaining Scope . . . 83
9.2 Remaining Modules and Features . . . 83
9.3 Remaining Iterations . . . 84
9.4 Implementation Plan . . . 84
9.5 Testing and Validation Plan . . . 85
9.6 Deployment Plan . . . 85
9.7 Tentative Timeline . . . 86
9.8 Expected Final Deliverables . . . 86

10 Conclusion **87**

A Proposal Action Register **88**
A.1 Why location matters: threat model . . . 88
A.2 Locality method comparison (design step A11) . . . 89
A.3 Locality test cases . . . 89
A.4 Privacy boundary in the built system . . . 89

B Detailed Requirements **92**
B.1 Dressed core use cases (summary) . . . 92
B.2 Requirement counts . . . 93

C Detailed Test Cases **94**

D Repository and Contribution Evidence **96**

E Additional Design Diagrams **99**
E.1 Class diagrams . . . 99
E.2 Sequence (b): handover and return to evidence . . . 99
E.3 Deployment as designed in Part A (historical) . . . 104
E.4 Wireframes . . . 104

F Meeting and Progress Evidence **110**

List of Figures

4.1 Core use case diagram (UML 2.5) 28

5.1 Layer diagram, Iteration 1 (UML package diagram, design step A8) . . . 34
5.2 Component diagram, Iteration 1 (UML 2.5, design step A8). Colour shows the owner (S1 blue, S2 orange, S3 purple) 35
5.3 Domain model v1.0 (UML class diagram without operations, design step A2) 44
5.4 ER diagram, schema v2.3 (crow's foot, design step A6): 22 tables, 217 columns, 48 foreign keys 46
5.5 Borrowing lifecycle state machine v2.2 (UML 2.5, design step A3) 47
5.6 Escalation case state machine (UML 2.5, design step A3) 48
5.7 Sequence (c): raise dispute, escalate, moderator decision (UC-B12, D6, I4) 49
5.8 Assurance pipeline, DFD level 0 (context diagram; Gane and Sarson, design step A10) 50
5.9 Deployment diagram, Iteration 1 as built (UML 2.5): API and web app on Render, PostgreSQL 16 on Neon, deployed from `main` after CI passes . 51

E.1 Class diagram, Student 1 packages: accounts and locality, lifecycle and event store, indirect trust 100
E.2 Class diagram, Student 2 packages: estimation, manipulation detection, decision policy and explanation 101
E.3 Class diagram, Student 3 packages: cases, decisions and audit (Module 9) and the scenario suite (Module 10) 102
E.4 Sequence (b): dual confirmation of handover and return, then outcome to the engine (UC-B7, B8) 103
E.5 Deployment diagram as designed in Part A (A13, provisional; superseded by the as-built deployment) 105
E.6 W1 lender request with assurance summary; W2 lender decision and date conflict 106
E.7 W3a handover inspection (lender); W3b handover confirmation (borrower) 107
E.8 W4 return inspection; W5 dispute form and progress 108
E.9 W6 moderator queue; W7 case adjudication 109

List of Tables

1.1 The ten modules and their state after Iteration 1 4
1.2 User classes 5
1.3 Measurable success criteria 5

2.1 Comparison with existing products and prior projects (proposal defence) . 9
2.2 Mapping to the Seoul Accord complex computing problem characteristics 11
2.3 How research findings shaped the design 12

3.1 Proposal action register (Form 10) 16
3.2 The four iterations 18
3.3 Work division for Iteration 1 (from the Iteration 1 plan) 20
3.4 Planned milestones against actual progress 21
3.5 Deviations from the Iteration 1 plan 22

4.1 Prioritised core use cases 23
4.2 UC-B5 Send borrowing request (condensed) 24
4.3 Core functional requirements and Iteration 1 status 25
4.4 Selected acceptance criteria and their tests 30
4.5 Traceability from requirement to design, code and tests 31

5.1 Design components mapped to the implementation 36
5.2 REST endpoints (Iteration 1) 38
5.3 Architecture decision records 39
5.4 Team decisions D10–D20 (13 October 2026) 40
5.5 Design changes since the proposal 41

6.1 Slices and their pull requests 53
6.2 Completed features 54
6.3 Key files, classes and functions 55
6.4 *k* sweep on the scale tier (SYNTHETIC data, config 1.2.0) 58
6.5 Decision rules in order (Algorithm 4) 59
6.6 Technology stack 61

7.1 Test levels 64
7.2 Representative test cases (input / expected / actual) 65

7.3 Backend results by area (14 October 2026) 67
7.4 Evaluation scenarios after Iteration 1 (SYNTHETIC data) 67
7.5 Defect log (Iteration 1) 70

8.1 Release milestones (merge times in Pakistan time) 74
8.2 GenAI use by module 76
8.3 Dependency register (versions as locked on 14 October 2026) 79
8.4 Risk register 81

9.1 Module roadmap, Iterations 2 to 4 83
9.2 Timeline 86

A.1 Candidate locality methods 89
A.2 Locality test cases (SYNTHETIC inputs) 90

B.1 Main success scenarios and key extensions of the core use cases 92

C.1 Automated test cases by suite 94

D.1 Merged pull requests (commit owners from the pull request's commits) . . 96

F.1 Recorded team meetings and reviews 110

List of Abbreviations

Abbreviation	Meaning
ADR	Architecture Decision Record
API	Application Programming Interface
AWS	Amazon Web Services
CCP	Complex Computing Problem
CD	Continuous Deployment
CI	Continuous Integration
CI (in statistics)	Credible Interval
CNIC	Computerised National Identity Card
CORE / ENAB / SUPP / STRC	Requirement scope tags: core, core-enabling, supporting, stretch
D1–D20	Team decisions recorded in `docs/design/00_management/`
DEI	Diversity, Equity and Inclusion
DF	Defect (defect log entries DF-01 to DF-09)
DFD	Data Flow Diagram
ER, ERD	Entity–Relationship (Diagram)
ES	Evaluation Scenario (ES-01 to ES-13)
EXIF	Exchangeable Image File Format (photo metadata)
FK	Foreign Key
FR	Functional Requirement
FYP	Final Year Project
GPS	Global Positioning System
HCI	Human–Computer Interaction
HTTP(S)	Hypertext Transfer Protocol (Secure)

Abbreviation	Meaning
JSON	JavaScript Object Notation
JWT	JSON Web Token
L2, L3	Levels of generative-AI use (assisted; core-assist), Section 8.7
MoSCoW	Must, Should, Could, Won't (prioritisation)
NFR	Non-Functional Requirement
NUL	The null character (code 0)
P2P	Peer to Peer
P50, P95	50th and 95th percentile (latency)
PKR	Pakistani Rupee
PR	Pull Request; in NFR identifiers, Privacy
R1–R9	Rules of the decision policy (Algorithm 4)
REST	Representational State Transfer
S1–S5	Manipulation signals; in ownership tables, Students 1 to 3
SDG	Sustainable Development Goal
SDK	Software Development Kit
SQL	Structured Query Language
SSD	System Sequence Diagram
SUS	System Usability Scale
TC	Test Case (Test Plan v1.3)
TF-IDF	Term Frequency–Inverse Document Frequency
TLS	Transport Layer Security
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
UTC	Coordinated Universal Time
W1–W7	Wireframes of the core screens

Chapter 1

Introduction and Project Vision

1.1 Project Overview

Digital Mohalla (*mohalla*: neighbourhood) is a neighbourhood borrowing and lending platform. A member lists an item, a neighbour requests it for some dates, the lender decides, the item changes hands, and it comes back. Every step is recorded, and the outcome of each loan becomes evidence for the next one.

Its central computing problem is to decide whether a proposed loan has enough trustworthy evidence when history is sparse, role-dependent and possibly manipulated, while protecting location privacy and sending uncertain cases to people. A recency-weighted Beta posterior, bounded indirect evidence and interpretable manipulation signals produce an explained *assurance state*; the platform never labels a person or issues a single trust score. Everything else in the platform (listings, workflow, search, interface) exists to feed that decision or to act on it.

1.2 Problem Statement

> Determine whether a proposed borrowing transaction has sufficient trustworthy evidence under sparse, role-dependent and potentially manipulated history, while protecting location privacy and escalating uncertain cases to humans.

Four properties make this hard:

1. **Sparse evidence.** Most neighbours have a handful of loans, or none. A newcomer with no history is not a risky person, so "no evidence" must not be reported as "bad evidence".

2. **Role dependence.** Returning items on time as a borrower says little about describing items honestly as a lender, so evidence is kept per role and metric.
3. **Indirect evidence.** A borrower may have a good record with the lender's neighbours but none with the lender. That evidence is useful but weaker than direct history and must never be enough on its own.
4. **Manipulation.** A small ring of accounts can rate one another up, and a lender can post a retaliatory review after a dispute. A real family that lends within itself looks statistically similar to such a ring, so signals must be evidence for a human, never an automatic verdict.

1.3 Problem Context and Motivation

Lending between neighbours is an old practice; what a platform has to add is a way to make a handover between near-strangers feel safe. In the team's own experience of Pakistani neighbourhoods, such lending is arranged informally, in messaging groups and by word of mouth, which leaves no record of who borrowed what, no evidence of the item's condition and no process for resolving a dispute. Research on online marketplaces shows that the trust cues a platform chooses change behaviour [1] and that averaged reputation hides uncertainty and is easy to game [2]. The project therefore puts its effort into the quality of the evidence behind a decision, not into another listings app.

1.4 Project Vision

A neighbourhood platform where any member can lend to any other member with a clear, honest picture of the evidence behind the request: how much verified history exists, how certain it is, what the community knows, whether anything looks anomalous, and which safeguards are sensible. The platform supports the lender's decision and never makes it for them; whenever the evidence is uncertain or suspicious, a human moderator decides.

1.5 Project Objectives

1. Record every borrowing transaction as an immutable, replayable event stream in which invalid transitions and double bookings are impossible (NFR-R-01 to R-03: 100% of runs).
2. Return, for every request, exactly one of four assurance states with its uncertainty, evidence, safeguards and explanation (FR-T-01), with direct-evidence assessment

under 1 s at the 95th percentile (NFR-P-03).

3. Show that the engine behaves as designed on the evaluation scenarios: at least 7 of 13 at mid-term and 13 of 13 at the end, state and explanation both correct (NFR-Q-01).
4. Route every hold and dispute to a moderator, never restrict anyone automatically, and record each decision with a reason in a tamper-evident log (FR-T-11, FR-M-05).
5. Keep exact location and identity numbers out of every API response and log (NFR-PR-01, NFR-S-07).
6. In FYP-2, show that the full engine beats global-average and role-average baselines on Brier score and calibration, with each component justified by ablation (NFR-Q-02).

1.6 Project Scope

The committed core is: community membership with privacy-preserving locality verification, borrowing requests, the event-sourced transaction lifecycle, evidence-based assurance, handover and return confirmation, dispute management and moderator review. Requirements are tagged CORE (named in the committed core), ENAB (core-enabling), SUPP (supporting) and STRC (stretch, only after the core works).

Excluded (or stretch only): vendors, community events, kameti (rotating savings), a general chatbot, a social feed, advertisements and broad analytics. Search and matching are supporting scope for Iteration 3. The project must not become a generic social network, a conventional listings app, a surveillance system, an opaque social-scoring system, or a large-language-model substitute for a computing challenge.

1.7 Project Modules

The platform is organised into the ten modules of the project overview. Table 1.1 gives each module's scope and its state after Iteration 1.

Table 1.1: The ten modules and their state after Iteration 1

#	Module	Scope	Iteration 1
1	User & Community Management	Core (membership, locality)	Password login and roles; communities, memberships and items seeded. Locality method designed (A11), build in Iteration 2
2	Community Events & Activities	Stretch	Not started (by design)
3	Resource Sharing & Item Management	Core (lifecycle)	**Built**: event store, 14-state lifecycle, request, accept/decline/cancel, handover, return, disputes, replay, double-booking guard
4	Search, Discovery & Matching	Supporting	Not started; Iteration 3
5	Trust, Reputation & Safety	**Core (CCP)**	**Built**: Beta posterior, similarity, trust edges, k-hop, two manipulation signals and the retaliation check, decision rules R1–R9 (R6 in two variants), explanations, engine-failure hold
6	Community Forum & Chatbot	Stretch	Not started (by design)
7	Communication & Notifications	Enabling	Not started; in-app notifications in Iteration 2
8	Vendor Management	Stretch	Not started (by design)
9	Administration & Moderation	Core (cases, disputes, audit)	**Built**: escalation cases with evidence trail, moderator decisions on disputes and holds, append-only hash-chained audit log; audit-log screen in Iteration 2
10	Analytics & Reports	Core (event logging, evaluation)	**Built**: frozen event logging, synthetic dataset generator, scenario suite in CI, k sweep

1.8 Stakeholders and User Classes

Table 1.2: User classes

User class	Characteristics and needs
Borrower	Requests items for dates; needs a fair hearing even with no history; may raise or withdraw a dispute.
Lender	Owns items; decides requests from the evidence summary; records the handover checklist and deposit (record only, no money moves).
Newcomer	A borrower with no verified history; must receive INSUFFICIENT_EVIDENCE with a route forward, never a negative label.
Moderator	A member trusted by one community; decides holds and disputes in that community only, always with a reason.
Admin	Manages communities, roles and categories (Iteration 2 onwards).
Community	The group of members who share a locality; the source of indirect evidence and the unit of moderation.
Project team and examiners	Need replayable events, versioned parameters and reproducible scenario results to evaluate the engine.

1.9 Success Criteria

Table 1.3: Measurable success criteria

Criterion	Measure	Status after Iteration 1
No double booking (NFR-R-01)	Exactly one of N simultaneous acceptances commits, every run	Met (TC-08: 5 simultaneous acceptances, 3 rounds)
Invalid transitions rejected (NFR-R-02)	100% of invalid (state, event) pairs refused	Met (TC-01: all 294 state–event pairs, 263 refused)

Criterion	Measure	Status after Iteration 1
Replay (NFR-R-03)	Rebuilt state equals stored state	Met (TC-07 and the replay tests of suite 2)
Tamper evidence (NFR-S-05)	100% of altered events detected	Met (TC-13, TC-14)
Configuration versioned (NFR-D-01)	100% of assessments store `config_version`	Met (TC-71, database NOT NULL)
Scenario quality (NFR-Q-01)	$\geq$7 of 13 at mid-term, 13 of 13 final	7 of 7 required pass; 11 of 13 states end to end
Assessment latency (NFR-P-03)	Direct evidence P95 $\leq$1 s	Not yet measured (Iteration 2); k-hop component only: P95 4.5 ms at $k = 3$
Privacy (NFR-PR-01, S-07)	No location or CNIC in any response or log	Met for the built API (suite 14)
Decision quality (NFR-Q-02)	Beats averaging baselines on Brier score and calibration	FYP-2 evaluation
Newcomer fairness (NFR-Q-07)	Assurance for new vs. established users reported	FYP-2 evaluation

1.10 Constraints and Assumptions

- Three students and two semesters: no framework or abstraction is added unless a requirement needs it.
- No real user data in FYP-1. All development and test data is synthetic, generated with a fixed seed and labelled synthetic in every file and screen. Public trust networks (Epinions, Bitcoin OTC) are used only for aggregate shape statistics; no rows are copied.
- The team does not run interviews or surveys; the location question is answered by a method comparison, threat model and test cases (Section 3.2).
- Free-tier hosting (Render and Neon); the API sleeps when idle, so the first request after a pause is slow.

- Every tunable parameter is an initial hypothesis kept in a versioned configuration file (currently `1.3.0-iteration1-draft`) and is to be justified by the FYP-2 sensitivity analysis.
- Android and web are the target clients; iOS is not targeted (team decision, 13 October).

1.11 Ethical, Legal and DEI Considerations

- **Decision support, not judgement.** Assurance states describe the evidence for a transaction, not the person. No screen shows a single trust number (NFR-U-01), and every state is shown with the evidence behind it.
- **No automatic restriction.** Manipulation signals are anomalies with evidence, worded neutrally; a request is only ever held for a moderator, who must give a reason. Nothing is restricted without a human decision.
- **Newcomers are not penalised.** Zero history yields INSUFFICIENT_EVIDENCE with the message "Insufficient verified history; lender review required" and sensible safeguards, never a low score. Fairness between new and established users is a reported metric (NFR-Q-07).
- **Privacy.** Only approximate (community-level) location is used; GPS is read once, in memory, when a member joins a community, and a borrowing request carries no location. No exact address or CNIC is stored. Automated tests check every API response model and log line.
- **Consent.** Pilot data in FYP-2 will be collected with consent and a right to withdraw (NFR-PR-05); applicable Pakistani data-protection obligations are to be confirmed with the supervisor.
- **Social value.** The project supports SDG 12 (responsible consumption, through reuse and local circulation) and SDG 11 (inclusive, safe communities).

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

The research had four strands, each feeding a specific design question:

1. **Literature on reputation and trust.** Beta reputation and its uncertainty [3, 4], surveys of trust and reputation systems [2, 5], trust propagation in social networks [6, 7], graph anomaly and collusion detection [8, 9, 10], and reciprocity in two-sided review systems [11, 12]. Each source is cited where it shaped a decision, and the report states whether a method was reproduced, adapted or replaced.
2. **Products and prior projects.** The products residents use today and four earlier FAST final-year projects in the same space were compared on the features that matter for safe lending (Section 2.3).
3. **Public data, for shape only.** Aggregate statistics of public trust networks (Epinions [13] and Bitcoin OTC/Alpha [14, 15]), such as degree distributions and the share of positive edges, were measured to make the synthetic neighbourhood data realistic. No rows from these datasets are copied into the project.
4. **Design-time verification.** Every algorithm was worked by hand on small examples (for instance an 8-node graph for the k-hop search and thirteen scenario fixtures for the decision policy) before code was written, and the code is tested against those worked numbers (Chapter 7).

2.2 Existing Solutions / Products

- **OLX and Facebook Marketplace** [16, 17]. General classifieds. Strong reach and listings, but built for one-off sales between strangers; trust cues are a profile and, at

most, a star average.

- **WhatsApp groups and kameti apps.** In the team's experience, the informal channel through which neighbourhood lending is arranged today. Familiar and immediate, but there is no record of who borrowed what, no condition evidence and no dispute process. Kameti (rotating savings) apps digitise contributions [18] but not item lending.
- **Nextdoor** [19]. A neighbourhood social network with verified addresses and posts; lending is informal and outside any workflow.
- **Peerby and Fat Llama** [20, 21]. Peer-to-peer rental platforms with listings and booking flows; reputation is a review average, without a statement of how much evidence stands behind it.
- **Prior FAST FYPs** (identified by their FAST FYP project IDs). Kiraya (F22-071) and AIRental (F23-029) address rentals, Kitabiyat (S25-040) book sharing and PawPal (F25-101) a pet community. None models evidence quality or manipulation.

2.3 Comparison of Existing Solutions

Table 2.1 repeats the comparison presented at the proposal defence. Ratings are the team's assessment of each product's public features at that time: ✓ supported, ∼ partial, × not supported.

Table 2.1: Comparison with existing products and prior projects (proposal defence)

Feature	DM	OLX/FB	WA/kameti	Nextdoor	Peerby/FL	Kiraya	AIRental	Kitabiyat	PawPal
P2P lending	✓	✓	∼	×	✓	∼	✓	✓	×
Shared reputation	✓	∼	×	∼	∼	×	×	×	∼
Trust-aware decisions	✓	×	×	×	∼	×	∼	∼	×
Moderation workflow	✓	∼	∼	∼	∼	×	×	×	∼
Unified neighbourhood platform	✓	×	×	✓	×	×	×	×	∼

DM = Digital Mohalla; FB = Facebook; WA = WhatsApp; FL = Fat Llama. The DM column shows the proposal-time vision: "Unified neighbourhood platform" and "Shared reputation" describe the full platform. After the defence the scope was narrowed (Section 1.7), and the engine reports evidence states for a loan rather than a reputation score for a person.

After the defence the comparison was narrowed to the properties the assurance engine adds, which none of the compared products offers as a whole: (1) reporting how much verified evidence stands behind a judgement, with an interval, instead of an average; (2) keeping evidence per role; (3) using the community's indirect evidence while capping it below direct evidence; (4) treating manipulation as a separate risk dimension reviewed by a person; and (5) recording handover, return and disputes as evidence for future decisions.

2.4 Limitations and Gaps

- **Averages hide uncertainty.** A 5.0 from one review and a 4.8 from two hundred look alike, and averaged reputation is easy to game [2, 5].
- **Newcomers are invisible or penalised.** Cold start cannot be solved by a model, because the history does not exist [22]; it needs an explicit policy.
- **No evidence trail.** Informal channels record nothing about handover condition or returns, so a dispute is one person's word against another's.
- **Manipulation is handled by deletion or not at all.** Where it is handled, it is usually an opaque automatic action, which is unacceptable in a neighbourhood where members cannot simply leave.

2.5 Proposed Solution

Digital Mohalla records every loan as an immutable event stream and turns confirmed outcomes into evidence. For each request the assurance engine (i) estimates the borrower's reliability in the borrower role with a recency-weighted Beta posterior and a 90% credible interval, (ii) adds capped indirect evidence from at most three hops in the lender's community, (iii) computes manipulation signals as a separate risk, and (iv) applies an ordered decision table that returns one of four states (SUFFICIENT_EVIDENCE, REVIEW_RECOMMENDED, INSUFFICIENT_EVIDENCE, ACTIVE_DISPUTE) with safeguards and an explanation. Requests the engine cannot clear are held for a moderator; nothing is rejected automatically.

2.6 Complex Computing Problem

The problem meets the Seoul Accord characteristics of a complex computing problem (Table 2.2).

2.7 Technical Challenges

1. **Concurrent acceptances.** Two lenders' (or one lender's two) acceptances for overlapping dates can race. Solved in the database: an exclusion constraint on held bookings makes the second commit fail, and the transition, booking and event commit in one transaction (ADR-0002).

Table 2.2: Mapping to the Seoul Accord complex computing problem characteristics

Characteristic	How Digital Mohalla meets it
Depth of knowledge	Bayesian inference with recency weighting; bounded graph search with path-diversity rules; anomaly detection on review graphs and text; concurrency control and event sourcing in a relational database.
Conflicting requirements	Safety vs. inclusion of newcomers; manipulation detection vs. false suspicion of real families; trust verification vs. location privacy; explanation detail vs. disclosure of other members' data.
Depth of analysis, no obvious solution	No off-the-shelf model answers "is there enough evidence for this loan"; the rule order, thresholds and caps had to be analysed on worked scenarios (ADR-0007, ADR-0008).
Infrequently encountered issues	Collusion in sparse graphs; retaliatory reviews after disputes; a ring that also trades with outsiders lowers its own density (camouflage [10]).
Beyond standards and codes	Reputation averages, the usual practice, are what the design replaces.
Diverse stakeholders	Borrowers, lenders, newcomers, moderators, admins and the community as a whole, with different and sometimes opposed interests.
Significant consequences	A wrong "sufficient" risks a physical item and a relationship between neighbours; a wrong "suspicious" damages a person's standing in a place they cannot leave.
Interdependence	The engine depends on lifecycle evidence (only confirmed outcomes count), and the lifecycle depends on the engine (holds, safeguards); disputes feed back into evidence after a moderator rules.
Ill-defined requirements	"Enough evidence" has no external definition; thresholds are hypotheses to be calibrated in FYP-2 against baselines and ablations.

2. **Correctness of the lifecycle under every command.** 14 states and 21 event types give 294 pairs; only 31 are valid. The transition table is enforced in Python and by a composite foreign key in PostgreSQL, and all 294 pairs are tested.

3. **Uncertainty under sparse data.** The posterior must distinguish "no evidence" from "bad evidence"; the effective evidence count n_{eff} and the interval's lower bound enter the rules directly.

4. **Correlated indirect evidence.** Several paths through the same neighbour are not independent; only the strongest path per first-hop neighbour is kept before combining with noisy-OR.

5. **Latency of graph search.** Path enumeration grows quickly with k; the $k = 1$ to 4 sweep (Section 6.7.2) measured a 3.5-fold rise in 95th-percentile latency from $k = 3$ to $k = 4$ for one percentage point more coverage.

6. **Real families vs. rings.** A family that lends within itself fills its review graph like a ring. A hold now needs two corroborating signals (ADR-0007), so one strong signal alone only asks for verification.

7. **Engine failure.** A slow or failing engine must never yield a default state; requests are held with reason `engine_failure` after a 2,000 ms timeout.

2.8 Research Findings and Design Decisions

Table 2.3: How research findings shaped the design

Finding	Source	Design consequence
Averaged reputation hides uncertainty and is easy to game	[2, 5]	Report a posterior with a 90% interval and effective evidence; never show a single number (NFR-U-01).
Binary outcomes have a conjugate Beta posterior	[3]	*Adapted*: Beta(1,1) prior updated with outcomes weighted by a 180-day half-life, kept per role and metric.

Finding	Source	Design consequence
Trust propagates with decreasing influence	[6]	*Adapted*: bounded k-hop paths with decay γ^{h-1}, one path per first hop, noisy-OR, capped at $\rho \leq 0.6$. A global score such as EigenTrust [7] was not used, because it gives every user one number.
Two-sided reviews are reciprocal by nature	[11, 12]	"A rated B and B rated A" was rejected as a signal; on the synthetic data it flagged 6 of 8 ordinary members (decision D18).
Collusion shows as groups that rate one another	[9, 10]	S1 measures how many of a member's review partners also reviewed each other both ways (partner clustering, D18).
Graph anomaly methods trade accuracy for interpretability	[8]	Rule-based, interpretable signals rather than a trained model: there is no labelled neighbourhood fraud data, and every signal must be explainable to a moderator.
Cold start is a policy problem	[22]	Zero history gives INSUFFICIENT_EVIDENCE with safeguards (rule R3), not a low score.
Trust cues change behaviour	[1]	States are worded as evidence, not as judgements of people; tests check that explanations contain no accusing words.
Explanations raise transparency and trust	[23]	Every assessment carries a written explanation naming the rule that fired and the evidence behind it.
Event logs make state reproducible and auditable	[24, 25]	Append-only, hash-chained event log; current state is a replay, and evaluation data is recorded from day one (FR-N-03).

Finding	Source	Design consequence
Exclusion constraints prevent overlapping ranges atomically	[26]	Double booking prevented in PostgreSQL with `btree_gist` (ADR-0002), not in application code.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The proposal was approved with minor modifications. The panel made three points:

1. the project should show that location is a real issue for neighbourhood lending;
2. the platform has positive social potential;
3. the technical challenge, as proposed, looked limited.

The third point had the largest effect. The proposal described a broad platform (listings, events, kameti, vendors, forum and chatbot, search and matching) with trust as one feature among many. After the defence the team made the trust decision the centre of the project: the complex computing problem is now stated as a single question (Section 1), stretch modules were moved behind the core, and Iteration 1 was spent on the assurance engine and the lifecycle that feeds it.

3.2 Proposal Action Register

Table 3.1 is the Form 10 register. Items 1–3 are the panel's points; items 4 and 5 are changes the team identified while planning against the handbook.

Table 3.1: Proposal action register (Form 10)

No.	Feedback / required change	Action taken	Status	Evidence
1	Show that location is a real issue	*Why location matters:* every loan ends in a physical handover, so the lender needs to know the borrower really lives in the community. Without a locality check, an outsider can join a neighbourhood to borrow and disappear, one person can join with several accounts, and a fake GPS fix can place anyone "inside" (threat model in A11). *Response:* five locality methods compared on false accepts, false rejects, privacy, spoofing resistance and cost; chosen: a one-time coarse geofence plus an invitation from a verified member, with a moderator deciding anything inconclusive (never an automatic rejection). The reference rule passes 12 checks in six test-case groups (spoofing, boundary homes, visitors, permission refusal, duplicate accounts, genuine residents), plus 4 invitation-error checks and 2 privacy and no-rejection checks (18 in total). A borrowing request now carries no location at all (decision D13); privacy tests check every API response and log.	Partly addressed: design evidence; method built in Iteration 2 (Module 1)	`docs/design/12_locality/`; `test_privacy.py`; Section A.2

No.	Feedback / required change	Action taken	Status	Evidence
2	Positive social potential	Kept as a design constraint: decision support only, newcomers never penalised, no automatic restriction, approximate location only. Aligned with SDG 11 and SDG 12.	Addressed	Chapter 1; FR/NFR Part F (framing note)
3	Limited technical challenges	Scope re-centred on the assurance engine as the CCP: recency-weighted Beta posterior, bounded k-hop indirect evidence, manipulation signals with corroboration, and an ordered decision policy, each with a written algorithm, worked examples and tests. Built and tested in Iteration 1; evaluation against baselines planned for FYP-2.	Addressed in Iteration 1; evaluation in FYP-2	Chapters 2, 5, 6; scenario suite in CI
4	Proposal §13 planned Iteration 1 as design only (team-identified)	The mid-evaluation brief asks for a live core module with CI/CD, so Iteration 1 was split into Part A (design) and Part B (three vertical slices, each deployable).	Done	Section 3.7
5	Proposal §14 listed trust as shared by all three students (team-identified)	Replaced by a module ownership split in which each student owns a technical part of the core (Table 3.3).	Done	Iteration 1 plan; commit history

3.3 Iteration Strategy

The project runs in four iterations, each ending in a working system and an evaluation (Table 3.2). Within Iteration 1, development followed vertical slices: each slice cuts through the database, domain, API and app, ends deployable, and is released from `develop` to `main` through a pull request.

Table 3.2: The four iterations

Iteration	Stage	Required outcome
1	FYP-1 Mid	Live core module implementing project logic; design evolved since the proposal; one failure case tested; CI/CD with meaningful branches and automated tests
2	FYP-1 Final	About 45% of committed scope; testing beyond the happy path; report matches the live system; written FYP-2 evaluation plan
3	FYP-2 Mid	All major modules connected; realistic scenarios; held-out attack generator; preliminary comparison with the FYP-1 baseline
4	FYP-2 Final	Evaluation (baselines, ablations, pilot), no regression, honest limitations, no new scope

3.4 Iteration 1 Plan

Objective: a live, tested core: the borrowing lifecycle and the assurance engine, with the human review loop around them.

Part A, design (steps A0–A14). Clean baseline; four then eight ADRs; domain model; lifecycle and escalation-case state machines with a transition table; data dictionary and event catalogue; algorithm and policy design with a configuration draft and hand-worked scenario expectations; ER diagram and schema; class, component and layer diagrams with a REST endpoint table; sequence and activity diagrams, SSDs and the use case diagram; DFD; locality design; wireframes; test plan and deployment diagram; and a design review (A14).

Part B, implementation. B0 setup (repository, CI, configuration loader, synthetic seed data), then:

- **Slice A, workflow skeleton:** event store, transition table, aggregate and replay, overlap constraint, authorisation on every transition, the request-to-completion loop

with a stub engine, minimal UI, first deployment. Gate: loop runs; matrix, replay, concurrency, authorisation and append-only tests pass.

- **Slice B, assurance core:** outcome extraction, Beta posterior, trust edges and k-hop, decision policy with safeguards and explanation, scenario suite in CI, lender evidence screen. Gate: ES-01, 02, 03, 06 and 13 pass with state and explanation.
- **Slice C, human loop:** manipulation checks (ES-04, ES-05, dense-group guard), disputes with freeze and evidence trail, moderator decisions for disputes and holds, privacy checks, moderator app. Gate: feature freeze.

Deliverables: the deployed system, the repository with CI/CD, the design baseline in `docs/design/`, this report, and the demo.

3.5 Iteration 2 Plan

Objective: complete the core modules and prepare the FYP-2 evaluation.

- **Module 1:** membership requests and approval (UC-A3, A4) with the A11 locality method; ES-09 and ES-10.
- **Module 3:** scheduler jobs (request expiry, no-show, reminders, overdue escalation), extensions and counter-offers; photo uploads with metadata stripping (decision D20).
- **Module 5:** all roles and metrics, reviews (UC-D1), the remaining signals (rating velocity S2, new-account activity S3, linkage S5); ES-07 to ES-12 with their workflows.
- **Module 7:** in-app notifications.
- **Module 9:** information requests, appeals and reversal; the audit-log screen with filters and masking.
- **Module 10:** simulator (50–100 users, 200–300 transactions), baselines, time-ordered hold-out, the written FYP-2 evaluation plan.

Evidence: all thirteen scenarios in CI, a first baseline comparison, the evaluation plan, and the report matching the live system.

3.6 Work Breakdown and Responsibilities

Table 3.3: Work division for Iteration 1 (from the Iteration 1 plan)

	S1 Fatima Khursheed: lifecycle, indirect trust, locality	S2 Areeba Javed: estimation, policy, detection	S3 Minahil Kashif: moderation, evaluation, apps
Modules	Module 3 lifecycle; Module 5 indirect trust; Module 1 login and locality design; CI	Module 5 trust estimation, decision policy, first manipulation checks	Module 9 cases and disputes; Module 10 scenario suite; resident and moderator apps; deployment
Part A	ADRs (lead), state machines, data dictionary, ERD, architecture, locality, indirect-trust part of A5, sequence (b)	A5 algorithm design (lead), config file, decision-policy activity diagram, sequence (a), DFD	Sequence (c), dispute activity diagram, SSDs, use case diagram, wireframes, test plan, deployment diagram
Part B	Repo and CI; Slice A lifecycle; Slice B trust edges and k-hop; Slice C privacy checks	Slice A engine interface; Slice B outcomes, posterior, policy; Slice C manipulation checks	Slice A UI; synthetic data; Slice B scenario suite and lender screen; Slice C disputes, moderator decisions and app
Tests owned	Transition matrix, replay, concurrency, authorisation, append-only, k-hop, k sweep, privacy	Beta maths, policy table, scenarios logic, dense-group guard, engine timeout	Dispute and hold resolution, UI tests, scenario suite

Module 5 is shared on purpose: S2 owns estimation, policy and detection, S1 owns indirect trust, and the boundary is one interface, `IAssess.assess(request)`. Module 10 sits with S3, so the Iteration 3 attack generator will not be written by the author of the manipulation checks. Feature pull requests are reviewed by a member other than their

main author: 22 of 24 before merge and the remaining two after merge (Section 8.5).

3.7 Progress Against Plan

Table 3.4: Planned milestones against actual progress

Milestone	Status	Evidence
Part A design baseline (A0–A14)	Done	`docs/design/`; design review A14 (11 Oct); decisions D1–D9 (11 Oct)
B0 setup	Done	PRs #1, #6, #7; CI on every push and pull request
Slice A: workflow skeleton	Done	PRs #2–#5; release PR #8
Deployment (CD)	Done	PRs #17, #20: API and web app on Render, database on Neon; releases #18, #24
Slice B: assurance core	Done	PRs #15, #16, #19, #21–#23, #25, #27, #28; releases #29, #30
Slice C: human loop	Done	PRs #31–#35; release #36 (feature freeze)
NFR-Q-01 at mid-term ($\geq$7 of 13)	Met	7 of 7 required scenarios pass with state and explanation
Freeze activities	Partly done	Report and test evidence done; release tag, alpha session and deployed-instance system tests outstanding (Section 3.8)

The design baseline was completed first (Part A documents dated 10 and 11 October 2026), and implementation followed it closely in an intensive build between 12 and 14 October 2026, with all three slices merged and released. The pace was possible because each task was specified in advance down to its test cases, and because an AI coding assistant drafted code under the team's direction (Section 8.7).

3.8 Delays, Deviations and Recovery Plan

Table 3.5: Deviations from the Iteration 1 plan

Deviation	Reason	Recovery
Locality method not built	Built only if the D-11 gate chose it; the slices took priority. Requests read the stored membership-time result (stub), so the lifecycle is unaffected	Built with membership (UC-A3, A4) in Iteration 2; ES-09, ES-10
Hosting: Render and Neon instead of one AWS host	The free tiers needed no server administration and deploy straight from `main` after CI; the provisional A13 design had marked AWS "to be confirmed"	Deployment diagram redrawn for the as-built target (Figure 5.9); rollback rehearsal before the FYP-1 final
Photo uploads (TC-59, TC-85) deferred	No upload storage yet; storing files safely needs size and type checks and EXIF stripping (decision D20)	Iteration 2; the `attachment` table already exists as designed
Audit-log screen (UC-I6) not built	The log itself is built (append-only, hash-chained, tamper-tested); the screen was the lowest-value Slice C item	Iteration 2 with filters and masking
Manipulation signals S2, S3, S5	Planned for Iteration 1: S1 and the retaliation check; ADR-0007 added S4; built: S1, S4 and the retaliation check	Iteration 2
Release tag `v0.1.0-rc1`, alpha session, deployed system tests (suite 15)	Not yet done at the time of writing	Before the demo: tag the release and run the demo story on the deployed instance (alpha session). Iteration 2: the timing tests, full-assessment latency (NFR-P-03) and transition latency (TC-61, NFR-P-04)

Chapter 4

Software Requirements

The requirements baseline is *FR/NFR v1.3* and *Use Cases v1.3* (11 October 2026), in `docs/design/01_requirements/`. Both use the same identifiers, one state list and one vocabulary. This chapter presents the core; the full set is in Appendix B.

4.1 User Interactions / Use Cases

The baseline has 68 use case IDs in ten groups (A accounts 9, B borrowing 13, C events 10, D trust 7, E forum 6, F chatbot 5, G communication 4, H vendors 5, I administration 6, J analytics 3). Eleven are core and fully dressed; Table 4.1 lists them with MoSCoW priority and their state after Iteration 1.

Table 4.1: Prioritised core use cases

Use case	Scope	Pri	FRs	Iteration 1
B5 Send borrowing request	CORE	M	FR-B-05 to B-07	Built
B6 Respond to request	CORE	M	FR-B-05, B-10	Built (no counter-offer)
B7 Confirm handover	CORE	M	FR-B-11, B-12	Built (photos in It. 2)
B8 Return and complete	CORE	M	FR-B-13 to B-15	Built
B12 Raise dispute	CORE	M	FR-B-16, B-17	Built
D3/4/5 Assess transaction assurance	CORE	M	FR-T-01 to T-07	Built

Use case	Scope	Pri	FRs	Iteration 1
D6 Escalate case with evidence trail	CORE	M	FR-T-10	Built
I4 Resolve escalated case or dispute	CORE	M	FR-T-11	Built (no appeal yet)
I6 View audit logs	CORE	M	FR-M-05	Log built; screen in It. 2
A3 Request community membership	CORE (It. 2)	M	FR-A-06 to A-09	Seeded; It. 2
A4 Approve membership	CORE (It. 2)	M	FR-A-10, A-11	Seeded; It. 2

4.2 Use Case Diagram

Figure 4.1 shows the Iteration 1 core. The Trust Engine is not an actor: it is part of the system, reached through the *include* from B5. A3 and A4 are drawn because they are core, although they are built in Iteration 2.

4.3 Detailed Use Cases

Table 4.2 gives UC-B5, the use case that runs the assurance engine; the other dressed use cases are summarised in Appendix B.

Table 4.2: UC-B5 Send borrowing request (condensed)

Primary actor: Borrower. **Secondary:** Lender, assurance engine. **Pre-condition:** the borrower is a verified member of the item's community and the item is published. **Success:** the request is ASSESSED (or HELD_FOR_REVIEW), its dates are marked as requested (soft hold, nothing blocked) and one event is appended per transition.	
1	The borrower opens an item and chooses dates and an optional message; the system checks the dates against held bookings.
2	The system creates the request (REQUEST_CREATED) and appends `request_created`.
3	The system reads the borrower's stored coarse locality result for the community and appends `locality_checked` (LOCALITY_CHECKED).

Step	Description
4	The system calls the assessment, stores the assurance state, evidence, signals, safeguards and `config_version`, and appends `assessment_generated` (ASSESSED).
5	The lender sees the request with its evidence summary.

Extensions. 2a Dates overlap a held booking: refused, next free window suggested. 3a Locality inconclusive or conflicting: HELD_FOR_REVIEW (`locality_inconclusive` / `locality_conflict`), never rejected. 4a No verified history: INSUFFICIENT_EVIDENCE, "Insufficient verified history; lender review required", still reaches the lender (ES-01). 4b Corroborated manipulation risk (level 4): HELD_FOR_REVIEW (`manipulation_risk`) and a case is opened; nothing is restricted (ES-04). 4c High value with weak or dissimilar evidence (rule R4): HELD_FOR_REVIEW (`high_value_low_confidence`) (ES-06). 4d Engine fails or exceeds 2,000 ms: HELD_FOR_REVIEW (`engine_failure`), failure logged, no state assigned.

4.4 Functional Requirements

Functional requirements are grouped by module. Table 4.3 lists the core (CORE-tagged) requirements of the modules worked on in Iteration 1 and their status. "Partial" names what is left.

Table 4.3: Core functional requirements and Iteration 1 status

ID	Requirement (shortened)	Iteration 1
Module 3: Resource Sharing (borrowing lifecycle)		
FR-B-05	No two held bookings for one item overlap; enforced by a database constraint at acceptance	Built
FR-B-06	Create a request with dates, soft hold, `request_created`	Built; duplicate and blocked-borrower checks in It. 2
FR-B-07	Locality check, then assessment, before the lender sees the request; holds never reject	Built (locality read from stored result)
FR-B-08	Enforce the single lifecycle; invalid transitions rejected and logged	Built
FR-B-09	Immutable events with the frozen schema; state derived by replay	Built

ID	Requirement (shortened)	Iteration 1
FR-B-10	Accept or decline; acceptance declines overlapping pending requests	Built; counter-offer, chat and expiry in It. 2
FR-B-11	Handover checklist and borrower confirmation; deposit recorded (record only)	Built; photos in It. 2 (D20)
FR-B-12	No-show after grace period; borrower may dispute condition	It. 2 (scheduler)
FR-B-13	Return, inspection, on-time or late outcome, close	Built
FR-B-14	Extensions and overdue escalation	It. 2
FR-B-15	Outcome to the engine, booking released, deposit release recorded	Built
FR-B-16	Raise a dispute in ACCEPTED, ACTIVE or RETURN_INITIATED; freeze; evidence trail	Built
FR-B-17	Withdraw restores the previous state; respond and no-response handling	Withdraw built; response handling in It. 2
FR-B-18	Idempotent scheduled jobs	It. 2
Module 5: Trust, Reputation and Safety (assurance engine)		
FR-T-01	Exactly one of four states with confidence, evidence, signals, safeguards, explanation	Built
FR-T-02	Recency-weighted Beta posterior per role, 90% interval; no history gives INSUFFICIENT	Built
FR-T-03	Weight by recency and similarity (category, value band, duration); confirmed outcomes only	Built
FR-T-04	Bounded k-hop evidence ($k = 3$, decay 0.7), capped; lifts sparse cases to REVIEW only	Built
FR-T-05	Manipulation risk 0–100, separate from trust, with evidence	Partial: S1, S4 and retaliation check; S2, S3, S5 in It. 2
FR-T-06	Context-aware policy; corroborated risk holds for a moderator, never restricts	Built
FR-T-07	Engine failure or timeout holds with `engine_failure`; no default state	Built
FR-T-08	Factors shown separately, never one number	Built for the lender screen; trust profile (D2) in It. 2

ID	Requirement (shortened)	Iteration 1
FR-T-09	Reviews after completed loans; retaliation flagged	Partial: retaliation check on recorded reviews; review submission in It. 2
FR-T-10	Escalation case with trigger, priority and evidence trail; reuse an open case	Built
FR-T-11	Moderator decides with a mandatory reason; dispute closes via `dispute_resolved`	Built; information requests and reversal in It. 2
Module 9: Administration and Moderation		
FR-M-05	Tamper-evident log of every privileged action and transition	Partial: append-only hash-chained log built; filters, masking and screen in It. 2
Module 10: Analytics (evaluation support)		
FR-N-03	Record every event needed for evaluation under the frozen schema	Built
Module 1: User and Community Management		
FR-A-04	Authenticate users and manage sessions	Partial: password login (argon2, JWT); lockout and reset in It. 2
FR-A-06 to A-11	Community selection, locality verification, membership approval	Designed (A11); It. 2

4.5 Non-Functional Requirements

Numeric targets are set only where Iteration 1 can measure them; the others are set in Iteration 2, once a baseline exists.

4.5.1 Reliability

- **NFR-R-01** With N simultaneous acceptances for overlapping dates, exactly one commits, in 100% of runs. *Met*: 5 parallel acceptances, 3 rounds (TC-08).
- **NFR-R-02** Invalid lifecycle transitions are rejected 100% of the time. *Met*: 263 of 263 invalid pairs refused in Python and by the database (TC-01).
- **NFR-R-03** Replay reproduces stored state exactly. *Met*: in Python and in SQL (`replay_state()`), TC-07.

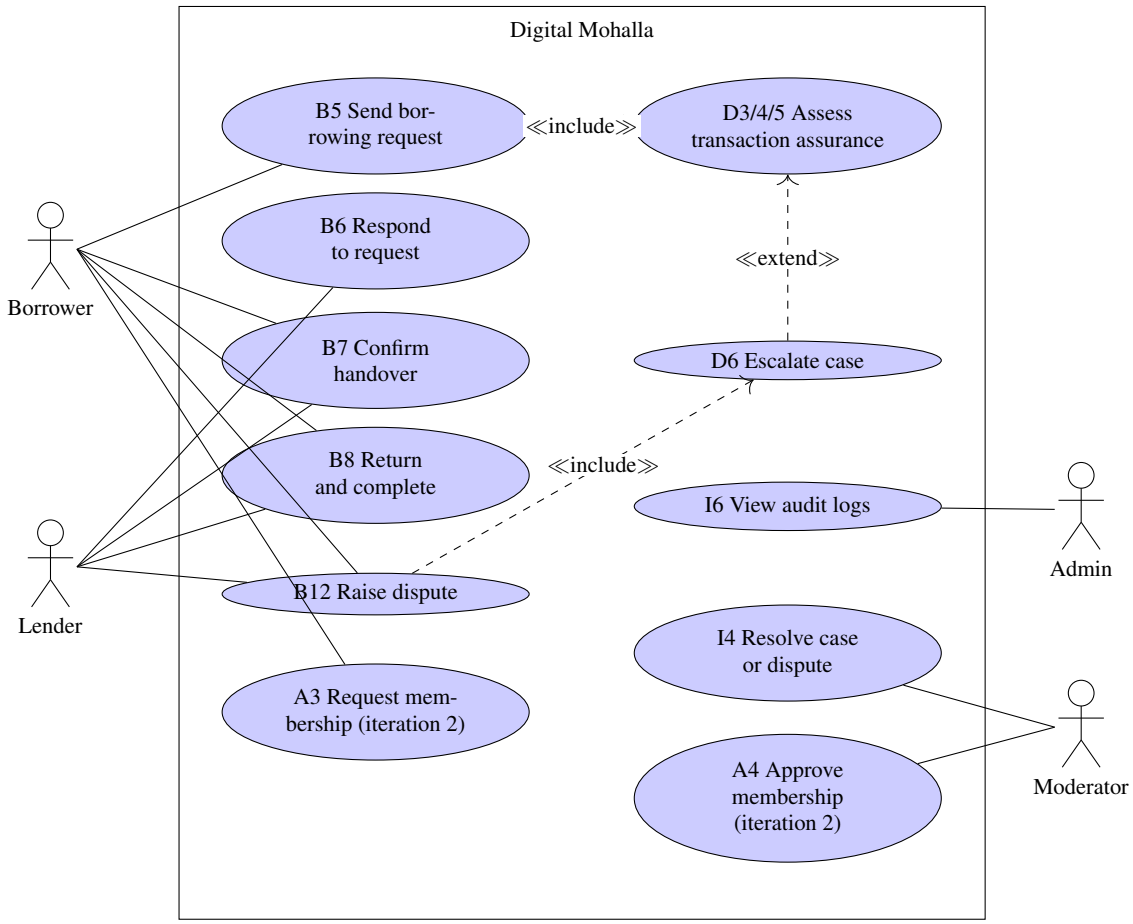


Figure 4.1: Core use case diagram (UML 2.5)

- **NFR-R-04** Engine failure routes to human review. *Met*: timeout and exception both hold with `engine_failure` (TC-36, TC-37).

4.5.2 Usability

- **NFR-U-01** Every screen with an assurance state shows its evidence, never one number. *Met* on the lender and moderator screens (TC-100, TC-107).
- **NFR-U-02, U-03** Task completion and SUS scores: measured with first-time users in Iteration 2.
- **NFR-U-05** Interface language: English in Iteration 1; Urdu is an open question.

4.5.3 Performance

- **NFR-P-03** Direct-evidence assessment P95 $\leq$1 s; k-hop latency reported for $k = 1$ to 4. *Not yet measured* for the full assessment, which will be timed in Iteration 2. Measured so far for the k-hop component only: P95 0.08, 0.71, 4.52 and 15.92 ms for $k = 1$ to 4 on the 2,026-user synthetic tier.
- **NFR-P-04** State transition commit P95 $\leq$500 ms. *Not yet measured*; TC-61 (suite 15) runs in Iteration 2.

4.5.4 Security

- **NFR-S-01** Passwords stored only as salted argon2 hashes; TLS for all traffic (Render terminates HTTPS).
- **NFR-S-02** Every transition checks and records the actor's authorisation; 100% of unauthorised attempts rejected and logged. *Met* (TC-10, audit entries for refused commands).
- **NFR-S-03** A moderator acts only in their own community. *Met* (moderation tests).
- **NFR-S-05** Audit and event logs are append-only and tamper-evident. *Met*: UPDATE and DELETE refused; an altered event is detected by `verify_event_chain()` (TC-13, TC-14).
- **NFR-S-07** No CNIC stored. *Met*: no such column; checked by the privacy suite.

4.5.5 Privacy, maintainability and decision quality

- **NFR-PR-01** Exact location never appears in profiles, listings, API responses or logs. *Met* for the built API: every response model and log line is scanned (suite 14).
- **NFR-D-01** 100% of stored assessments record `config_version`. *Met* (TC-71; the column is NOT NULL).
- **NFR-D-04** Automated tests run on every build and a failing test blocks merge. *Met*: three CI jobs on every pull request.
- **NFR-Q-01** At mid-term, $\geq$7 of 13 scenarios correct in state and explanation. *Met*: 7 of 7 (Section 7.4.1).

4.6 Acceptance Criteria

25 of the 35 core FRs have a Given/When/Then criterion in FR/NFR Part E. Table 4.4 shows a selection with the test that checks it.

Table 4.4: Selected acceptance criteria and their tests

FR	Given	When	Then	Test
FR-B-05	Two ASSESSED requests with overlapping dates	Both lenders accept at the same moment	Exactly one is ACCEPTED; the other ends DECLINED "dates taken"	TC-08
FR-B-08	A booking in COMPLETED	A transition to ACCEPTED is attempted	Rejected (409) and logged with the reason	TC-02
FR-B-09	A transaction with N events	Its state is rebuilt by replay in `stream_seq` order	The rebuilt state equals the stored state	TC-07
FR-B-16	An ACTIVE loan	A party raises a dispute with a reason	DISPUTED, previous state stored, transitions frozen, case opened	TC-50
FR-B-17	An open dispute	The raising party withdraws	`dispute_withdrawn` restores the previous state	TC-51
FR-T-02	A user with no verified history	The assessment runs	INSUFFICIENT_ EVIDENCE with Beta(1,1) and a wide interval	TC-20, TC-40

FR	Given	When	Then	Test
FR-T-03	Outcomes aged 10 and 360 days, half-life 180 days	Weights are computed	About 0.96 and 0.25; unconfirmed outcomes excluded	TC-22, TC-23
FR-T-04	Two paths of at most 3 hops via different first hops	Indirect evidence is computed	Reliability ≤ 0.6, decay 0.7^{h-1}, longer paths ignored	TC-25, TC-26
FR-T-06	Strong trust and corroborated risk ≥ 60	The policy runs	HELD_FOR_REVIEW (`manipulation_risk`), case opened, nothing restricted	TC-43
FR-T-07	An injected engine timeout	A request is assessed	HELD_FOR_REVIEW (`engine_failure`), failure logged, no state	TC-36
FR-T-11	An open dispute and an authorised moderator	The moderator decides with a reason	`dispute_resolved` completes the loan; the resolved outcome becomes evidence	TC-53
FR-M-05	Any transition	It is performed	An audit entry exists; altering it is detected	TC-14

4.7 Requirements Traceability

Table 4.5: Traceability from requirement to design, code and tests

Requirement	Design	Code	Tests	Owner
FR-B-05	ADR-0002; ERD `booking`	`schema` EXCLUDE; `service.accept`	`test_borrowing_service` (TC-08, TC-15)	S1
FR-B-08	State machine; transition table	`transition_table.py`; FK to `allowed_transition`	`test_transition_table` (TC-01, TC-02, TC-05)	S1
FR-B-09	ADR-0005; event catalogue	`eventstore/store.py`; `aggregate.py`	`test_event_store`, `test_aggregate` (TC-07)	S1

Requirement	Design	Code	Tests	Owner
FR-B-16, B-17	Sequence (c); ADR-0004	`service.raise_dispute`, `withdraw_dispute`	`test_disputes`, `test_api_disputes` (TC-50–52)	S3
FR-T-02, T-03	Algorithm 1	`estimation/posterior.py`, `similarity.py`	`test_beta_posterior`, `test_posterior_properties`, `test_similarity` (TC-20–24, TC-77)	S2
FR-T-04	Algorithm 2; ADR-0006, 0008	`indirect/khop.py`, `edges.py`	`test_khop`, `test_trust_edges`, `test_khop_on_dataset` (TC-25–28)	S1
FR-T-05	Algorithm 3; ADR-0007; D18, D19	`detection/manipulation.py`	`test_manipulation` (TC-30, TC-38)	S2
FR-T-06, T-07	Algorithm 4; activity diagram	`policy/decision.py`; `service` timeout	`test_decision_policy` (TC-29–39)	S2
FR-T-01, T-08	Explanation design; W1	`policy/explanation.py`; `AssurancePanel`	`test_explanation`, `lender-evidence-test` (TC-75, TC-100)	S2, S3
FR-T-10, T-11	Sequence (c); ADR-0003; escalation state machine	`moderation/decisions.py`; `service.decide`	`test_moderation`, `test_moderation_rules` (TC-53–57)	S3
FR-M-05	ERD `audit_log`	hash chain triggers; `AuditLog` port	`test_event_store` (TC-13, TC-14)	S1
NFR-PR-01	DFD privacy boundary; D13	`StrictIn` request bodies	`test_privacy`, `test_fuzz` (TC-58, TC-60, TC-84)	S1
NFR-Q-01	Scenario fixtures	engine end to end	`tests/scenarios/` (TC-40–47)	S3

Chapter 5

System Design

The design baseline was produced in Part A (steps A1–A14) and lives in `docs/design/`, one folder per artefact kind. Diagrams follow their standard notation (UML 2.5; crow's foot for the ER diagram; Gane and Sarson for the DFD) and were reviewed against those notations in the A14 design review. Each figure below is followed by the reasoning behind it and the requirements it serves. Where the built system differs from a Part A diagram, the difference is stated and justified in Section 5.5.

5.1 System Architecture

Digital Mohalla is a **modular monolith**: one FastAPI service with strict internal layers, one PostgreSQL 16 database, and a React Native (Expo) client that runs on Android and the web. Figure 5.1 shows the layers.

Reasoning. Each upper layer depends only on the one below it, and infrastructure depends upward on the ports it implements. The *domain* layer (lifecycle model, assurance engine, moderation rules) is pure Python with no input/output, so the trust maths and the decision rules can be unit-tested without a database (477 unit tests run in seconds). Repositories are declared as interfaces in the application layer and implemented in infrastructure (dependency inversion), which is why the engine reaches the database only through the `EvidenceSource` interface. A monolith was chosen over services because three developers and one database do not need network boundaries, and a single transaction must cover the lifecycle transition, the booking and the event (ADR-0002). Cross-cutting policies (authorisation on every transition, the frozen event schema, `config_version` on every assessment, location privacy, synthetic labels) apply to every layer. *Serves:* NFR-D-01, NFR-D-04, NFR-S-02, NFR-PR-01. *Owner:* S1. The platform layer shows the deployed platform, Render and Neon (Section 5.8.5); the provisional AWS host of Part A is recorded in Section 5.5.

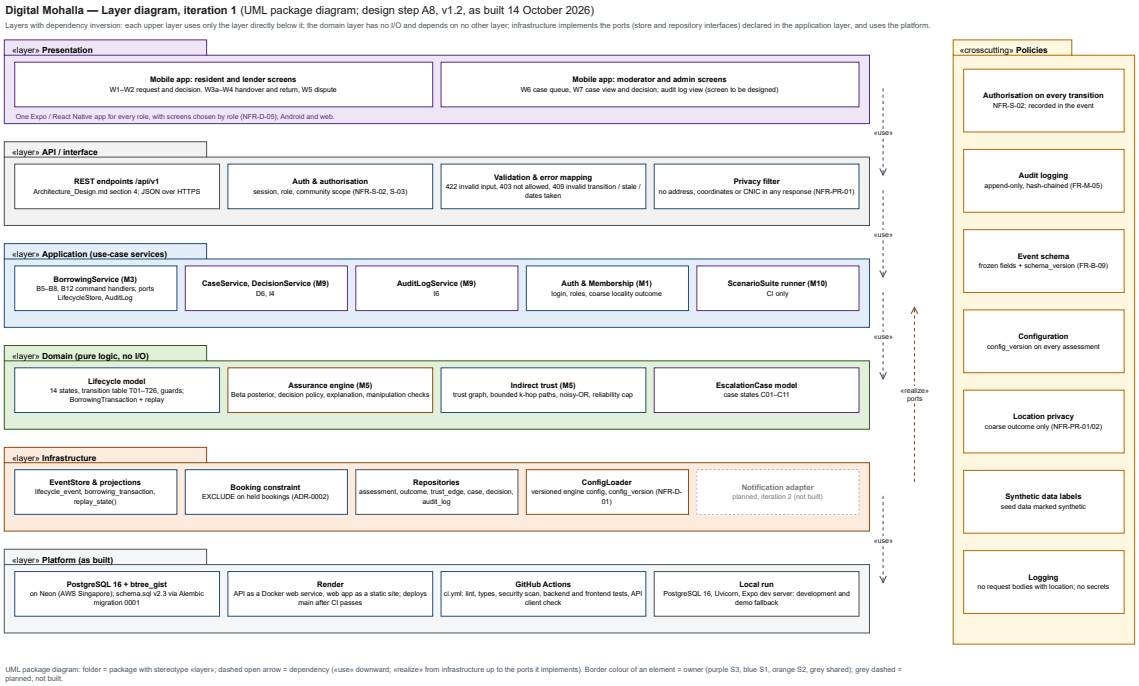


Figure 5.1: Layer diagram, Iteration 1 (UML package diagram, design step A8)

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/03_architecture/layer_diagram.drawio`

5.2 Architectural Components

Figure 5.2 shows the ten modules, with the five worked on in Iteration 1 in detail.

Reasoning. The diagram makes ownership and boundaries visible. Module 3 (lifecycle) and Module 5 (assurance) meet at exactly one provided interface, `IAssess.assess(request)`; inside Module 5 the S1/S2 boundary is the indirect-evidence interface. Module 3 calls the engine once per request, so a slow or failing engine affects one request and is turned into a hold (FR-T-07) instead of an error. Moderation (Module 9) changes the lifecycle only through lifecycle commands, so a moderator's decision passes the same transition checks as any other actor's. Modules 2, 4, 6 and 8 are drawn as deferred so the scope boundary is explicit. *Serves:* FR-T-01, FR-T-07, FR-T-11, FR-B-08.

From design components to code. During implementation several design classes became plain functions or merged into one service, because Python modules and protocols gave the same boundaries with less code. Table 5.1 maps every component in Figure 5.2 to the code that realises it.

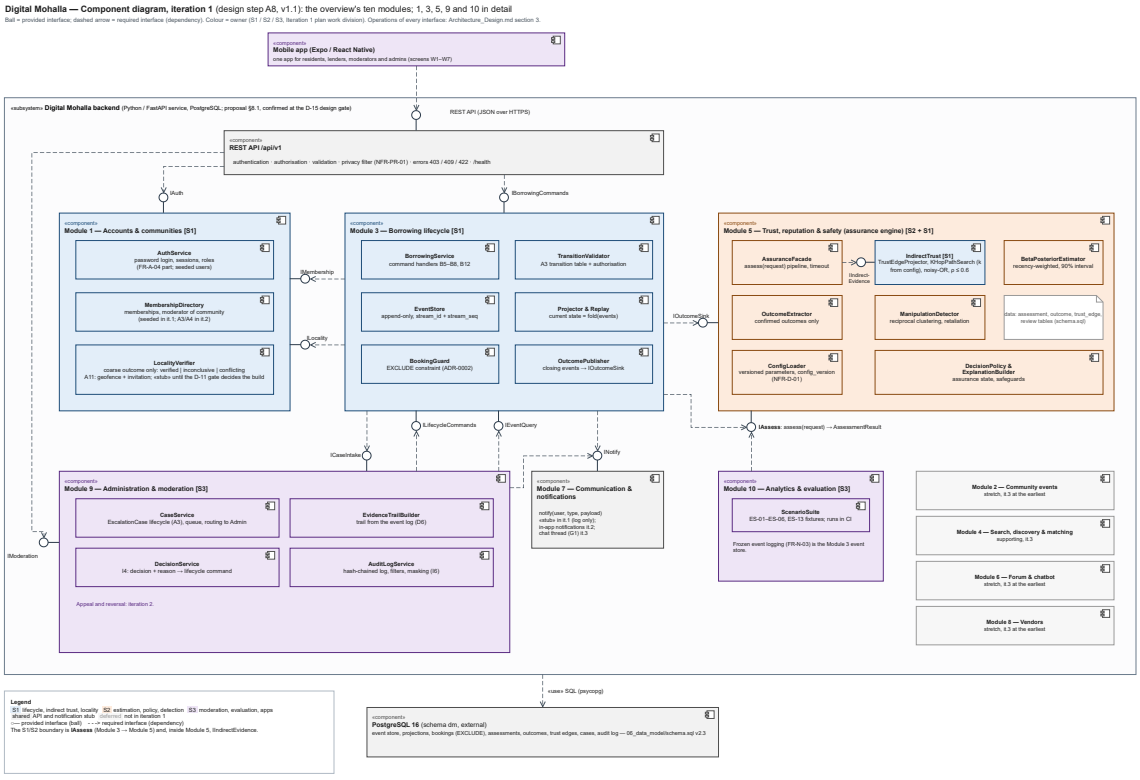


Figure 5.2: Component diagram, Iteration 1 (UML 2.5, design step A8). Colour shows the owner (S1 blue, S2 orange, S3 purple)

Full-size diagram: Open the full-size diagram in draw.io

Source: `docs/design/03_architecture/component_diagram.drawio`

Table 5.1: Design components mapped to the implementation

Design element (A7/A8)	Implementation	Note
`IAssess`	`IAssess` protocol, `domain/lifecycle/ports.py`	As designed
`AssuranceFacade`	`AssuranceEngine.assess()`, `domain/assurance/engine.py`	Renamed
`BetaPosteriorEstimator`	`estimate()`, `credible_interval()`, `recency_weight()` in `estimation/posterior.py`; `similarity.py`	Functions instead of a class (pure)
`OutcomeExtractor`, `IOutcomeSink`	`estimation/outcomes.py` (rules) and `infra/db/outcomes.py` (writer), called from the close hook in `BorrowingService`	Interface replaced by a direct, idempotent call on loan close
`IndirectTrust`, `IIndirectEvidence`, `TrustEdgeProjector`, `KHopPathSearch`	`indirect/khop.py` (`find_paths`, `indirect_trust`), `indirect/edges.py`, `infra/db/trust_edges.py`	S1/S2 boundary kept as a module boundary
`ManipulationDetector`, `ReciprocalClustering`, `ReviewSimilarity`, `RetaliationCheck`	`detection/manipulation.py`: `partner_clustering`, `review_similarity`, retaliation set-aside, `combine`	S1 redefined as partner clustering (D18)
`DecisionPolicy`, `ExplanationBuilder`	`policy/decision.py` (`decide`, `RULES`), `policy/explanation.py`	As designed
`ConfigLoader`	`config/engine_config.py`, `config/engine.json`	As designed
`BorrowingService`	`BorrowingService`, `domain/lifecycle/service.py`	Also hosts dispute and moderation commands
`TransitionValidator`	`TransitionTable` (`transition_table.py`) and the database foreign key to `allowed_transition`	Checked twice

Design element (A7/A8)	Implementation	Note
`EventStore`, `IEventQuery`, `Projector & Replay`	`LifecycleStore` protocol; `infra/eventstore/postgres.py`, `store.py`; `aggregate.py`; SQL `replay_state()`	As designed
`BookingGuard`	PostgreSQL exclusion constraint on `booking`	No Python class needed
`CaseService`, `DecisionService`, `ICaseIntake`, `IModeration`	`domain/moderation/decisions.py` (`plan`) and `BorrowingService.decide`, `moderation_queue`, `moderator_case`	Merged into the lifecycle service
`EvidenceTrailBuilder`	`BorrowingService.dispute_case`, `moderator_case` (trail read from `lifecycle_event`)	As designed
`AuditLogService`	`AuditLog` protocol; hash-chained `audit_log` table	Screen in Iteration 2
`LocalityVerifier`, `ILocality`	`LifecycleStore.locality_outcome()` reading the stored result (`locality_outcome_for()`)	Stub; method in Iteration 2
`INotify`	Not built	Iteration 2
`AuthService`, `IAuth`	`domain/accounts/auth.py`, `api/deps.py`	Minimal login

5.3 Interfaces and Dependencies

5.3.1 External interface: REST API

The API is served under `/api/v1`, takes and returns JSON, and is described by an OpenAPI document from which the app's typed client is generated (a CI job fails if the two drift apart). Every request body refuses unknown fields, so a client cannot set the actor, the authorisation or the next state itself.

Errors follow one mapping: 401 without a valid token, 403 for the wrong actor, 404 for data the member may not see, 409 for an invalid transition, a stale command or "dates taken", and 422 for invalid input.

Table 5.2: REST endpoints (Iteration 1)

Method	Path	Purpose
POST	`/auth/login`	Password login; returns a token
GET	`/items`, `/items/{id}`	Items visible to the member
POST	`/requests`	Create a borrowing request (UC-B5)
GET	`/requests`, `/requests/{id}`, `/lender/requests`	The member's requests; one request with its assessment; the lender's queue
POST	`/requests/{id}/accept`, `/decline`, `/cancel`	Lender decision; borrower cancellation (UC-B6)
POST	`/transactions/{id}/handover/checklist`, `/handover/confirm`	Handover (UC-B7)
POST	`/transactions/{id}/return`, `/return/confirm`	Return (UC-B8)
GET	`/transactions/{id}/events`	The event trail
POST, GET	`/transactions/{id}/dispute`, `/dispute/withdraw`	Raise, view and withdraw a dispute (UC-B12)
GET	`/moderation/cases`, `/moderation/cases/{id}`	Moderator queue and case view
POST	`/moderation/cases/{id}/decision`	Moderator decision with reason (UC-I4)
GET	`/health`	Liveness check for deployment

5.3.2 Internal interfaces

- `IAssess.assess(AssessmentRequest) -> AssessmentResult`: the only way the lifecycle reaches the engine; called with a 2,000 ms timeout read from the configuration.
- `LifecycleStore`: everything the lifecycle service needs from storage (create, append, view, assessments, cases, decisions, bookings), implemented on PostgreSQL.
- `EvidenceSource`: what the engine reads (the borrower's outcomes, the trust graph, open and recent disputes, verification, reviews and complaints), so the engine itself has no database code.
- `AuditLog`: records refused commands and privileged actions.

External dependencies are listed with versions and licences in Section 8.8.

5.4 Architecture Decisions and Rationale

Eight ADRs record the decisions that shape the code. The hosting choice (Render and Neon instead of the provisional AWS host) is recorded as a design change (Table 5.5); its ADR is pending. (`docs/design/03_architecture/DigitalMohalla_ADRs_Ver1.md`).

Table 5.3: Architecture decision records

ADR	Decision	Rationale
0001	Soft hold: pending requests mark dates as requested and block nothing	A lender should see all interest; blocking at request time would let one borrower lock an item with requests
0002	The database enforces the no-overlap rule at acceptance (exclusion constraint on held bookings); transition, booking and event commit in one transaction	Replaces "lower `stream_seq` wins", which only orders events within one transaction and cannot arbitrate between two transactions
0003	Exits from HELD_FOR_REVIEW depend on the hold reason (new event `hold_cleared`)	A cleared locality hold must still be assessed; a cleared manipulation hold already has an assessment; an engine-failure hold has nothing to attach safeguards to

ADR	Decision	Rationale
0004	Separate `dispute_resolved` and `dispute_withdrawn` events	A withdrawal restores the previous state; a resolution completes the loan and creates evidence; one event type could not mean both
0005	Event-specific data goes in a `payload` field defined per event type	Keeps the frozen schema stable while each event carries its own data (dates, reasons, safeguards)
0006	A trust edge's weight is the recency-weighted Beta posterior mean of the source's verified outcomes with the target (experimental)	One estimator for direct and indirect evidence; never a manual vouch
0007	A manipulation hold needs corroboration (risk $\geq$60 and two signals $\geq$0.5); S4 active in Iteration 1	"$\geq$60 always holds" contradicted the false-positive guard for real families; with one signal ES-04 could not be held at all
0008	Community paths lift sparse direct evidence from INSUFFICIENT to REVIEW_ RECOMMENDED, never to SUFFICIENT	Without it the indirect evidence could never change a decision

Team decisions record the smaller choices: D1–D9 on 11 October (for example the `payload` field, no disputes from RETURNED, merging the Request table into the transaction) and D10–D20 on 13 October (Table 5.4).

Table 5.4: Team decisions D10–D20 (13 October 2026)

#	Topic	Decision
D10	Trust-edge weight	ADR-0006 accepted as experimental
D11	Outcome mapping	Borrower: on-time return, condition agreed, dispute-free; lender: condition agreed, dispute-free; a party found at fault fails the relevant metrics

#	Topic	Decision
D12	SUFFICIENT_EVIDENCE thresholds	Mean ≥ 0.75, lower 90% bound ≥ 0.60, $n_{\text{eff}} \geq 3$, $\rho_{\text{sup}} = 0.3$: initial hypotheses
D13	ES-05 expected result	R8, SUFFICIENT_EVIDENCE with safeguards: the retaliatory review is detected and set aside
D14	ES-13 expected result	R9, REVIEW_RECOMMENDED; without the community paths, R6_SPARSE
D15	Device scope	Android only for device testing; iOS not targeted
D16	Test plan use	Tools added only when a test first needs them
D17	Similarity of a past loan	$0.4\times$ value-band closeness $+0.4\times$ same category $+0.2\times$ duration closeness, floor 0.5 (S2)
D18	Meaning of S1	Partner clustering instead of pairwise reciprocity (confirmed by S2, 14 October)
D19	Effect of a retaliation flag	The review is set aside from S1 and S4 and explained; reviews never enter the trust estimate (confirmed by S2, 14 October)
D20	Photo uploads	Deferred to Iteration 2 (confirmed by S1 and S2, 14 October)

5.5 Design Evolution

Table 5.5: Design changes since the proposal

Proposal / earlier design	Now	Why
Nine technical components M1–M9 across a broad platform	Ten modules; the assurance engine is the core, stretch modules deferred	Panel point 3; the handbook's scope test needs a technical core per student

Proposal / earlier design	Now	Why
Trust shared by all three students	S1 indirect trust, S2 estimation/policy/detection, S3 moderation/evaluation	Each member owns a removable technical part
FAISS search, learning-to-rank, LightGBM	Search in Iteration 3; detection is rule-based and interpretable	No labelled data for a trained detector; every signal must be explainable to a moderator
Four reputation tiers	Four assurance states that describe evidence, not people	A tier labels a person; a state describes this loan's evidence
"Lower `stream_seq` wins" for double booking	Database exclusion constraint (ADR-0002)	Correct across transactions and atomic
"RESUME" to end a dispute	`dispute_resolved` / `dispute_withdrawn` (ADR-0004)	Different outcomes need different events
Separate Request table	Merged into `borrowing_transaction` (D7)	One stream per loan from request to close
Any risk $\geq$60 holds	Two corroborating signals needed (ADR-0007)	Protects real families (C3 guard)
Indirect evidence with no effect on the decision	R6_SPARSE / R9 (ADR-0008)	Makes ES-13 decidable
Pairwise reciprocity signal	Partner clustering (D18)	Two-sided reviews are reciprocal by nature; the pairwise rule flagged 6 of 8 ordinary members
GPS sent with every request	Request carries no location; membership-time result is read (D13)	Privacy boundary moves to joining a community

Proposal / earlier design	Now	Why
One AWS host with Docker Compose (A13, provisional)	Render (API container and static web app) with Neon PostgreSQL	No server administration; deploys from `main` after CI
Design interfaces such as `IOutcomeSink`, `IIndirectEvidence`	Four protocols (`IAssess`, `LifecycleStore`, `EvidenceSource`, `AuditLog`)	Same boundaries, less code (Table 5.1)

5.6 Domain Model

Reasoning. The central class is the *BorrowingTransaction*, whose state is derived from its *LifecycleEvents*; an *Assessment* is attached to a transaction, not to a user, because an assurance state describes the evidence for one loan. *Outcome* is separate from *Review*: trust is computed only from verified outcomes, and reviews feed only the manipulation signals (D19), which is why a retaliatory review cannot lower anyone's trust. *Booking*, *ConditionReport* and *LocalityResult* were kept as classes (integration decision of 11 October) because each is realised by a table and guarded by a database rule. *Serves:* FR-B-09, FR-T-01, FR-T-03, FR-T-09.

5.7 Data Design

Reasoning. The schema (`schema.sql` v2.3, applied by Alembic migration `0001`) enforces the rules that must never depend on application code:

- **Event store.** `lifecycle_event` is append-only (a trigger refuses UPDATE and DELETE) and hash-chained (`prev_hash`, `row_hash`); `verify_event_chain()` detects any altered row. `(stream_id, stream_seq)` is unique.
- **Valid transitions.** `(previous_state, event_type, new_state)` is a composite foreign key to `allowed_transition`, so an invalid transition cannot be stored even if the Python check had a bug. The trigger `event_validate` checks the actor and sequence; `event_apply` updates the projection.
- **Bookings.** `EXCLUDE (item_id WITH =, period WITH &&) WHERE held`, using

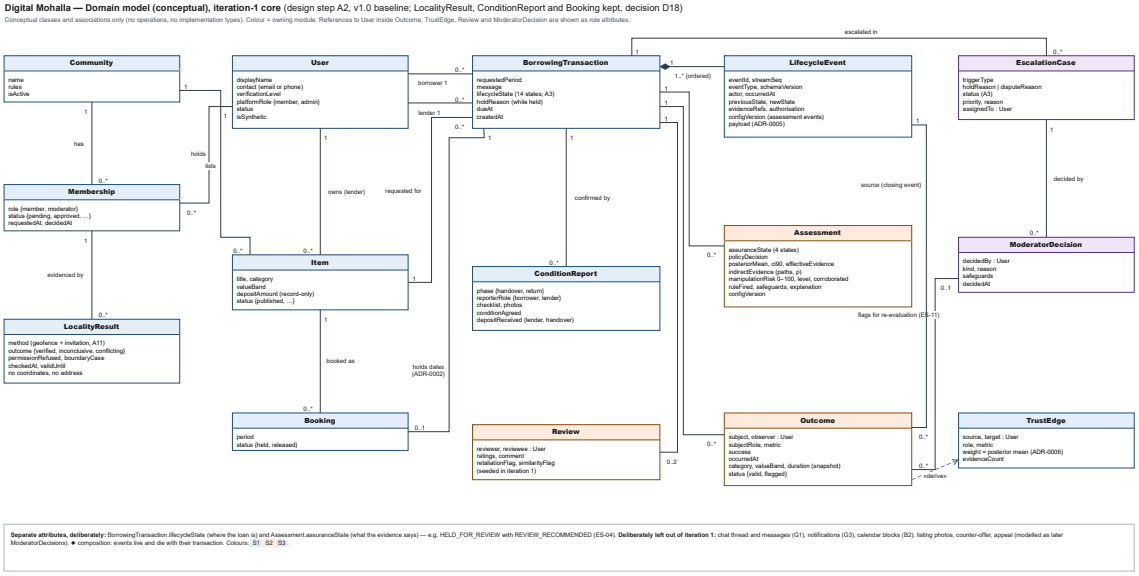


Figure 5.3: Domain model v1.0 (UML class diagram without operations, design step A2)

Full-size diagram: Open the full-size diagram in draw.io

Source: `docs/design/05_class_model/domain_model.drawio`

btree_gist.

- **Assessments.** Every row stores config_version (NOT NULL); assessment_rule_ck refuses a row whose rule, state, decision and hold reason disagree with the decision table.
- **Privacy.** No column stores coordinates, an address or a CNIC; a test scans the catalogue for such columns.

The current state is a projection (borrowing_transaction.current_state) that replay_state() can rebuild from the events alone. *Serves:* FR-B-05, FR-B-08, FR-B-09, FR-M-05, NFR-D-01, NFR-PR-01. *Owner:* S1 (lifecycle tables), S2 (assessment, outcome, trust edge, review fields).

5.8 Process / Behavior Design

5.8.1 Borrowing lifecycle

Reasoning. Fourteen states in two composites: PENDING (dates only requested) and IN_PROGRESS (booking held). Transitions that apply to every substate of a composite are drawn once on its border, and DISPUTED returns through a shallow-history state to the state stored in dispute_opened. HELD_FOR_REVIEW is never an automatic rejection: each hold reason has its own exits (ADR-0003). With the 21 event types this gives 294 state–event pairs, of which 31 are valid; the 31 valid pairs yield 34 leaf transitions because some events have guarded alternatives (for example hold_cleared returns to LOCALITY_CHECKED or ASSESSED depending on the hold reason). Transitions marked "it.2" (expiry, no-show, extensions) are part of the table but are triggered by the Iteration 2 scheduler. *Serves:* FR-B-08, FR-B-16, FR-B-17, FR-T-07.

5.8.2 Escalation case

Reasoning. A case opens for a dispute or a hold and ends either decided (with a mandatory reason) or withdrawn without a decision when the loan leaves the hold or dispute, for instance when the borrower cancels or withdraws (D9). A second dispute or hold on the same loan attaches to the open case instead of creating another (FR-T-10). A decision is stored append-only in moderator_decision and then applied as a lifecycle event in the hash-chained event log. Iteration 1 builds opening (C01), deciding (C06) and withdrawal (C07); information requests (C02–C04) and the appeal and reversal path (C08–C11) are built in Iteration 2. *Serves:* FR-T-10, FR-T-11.

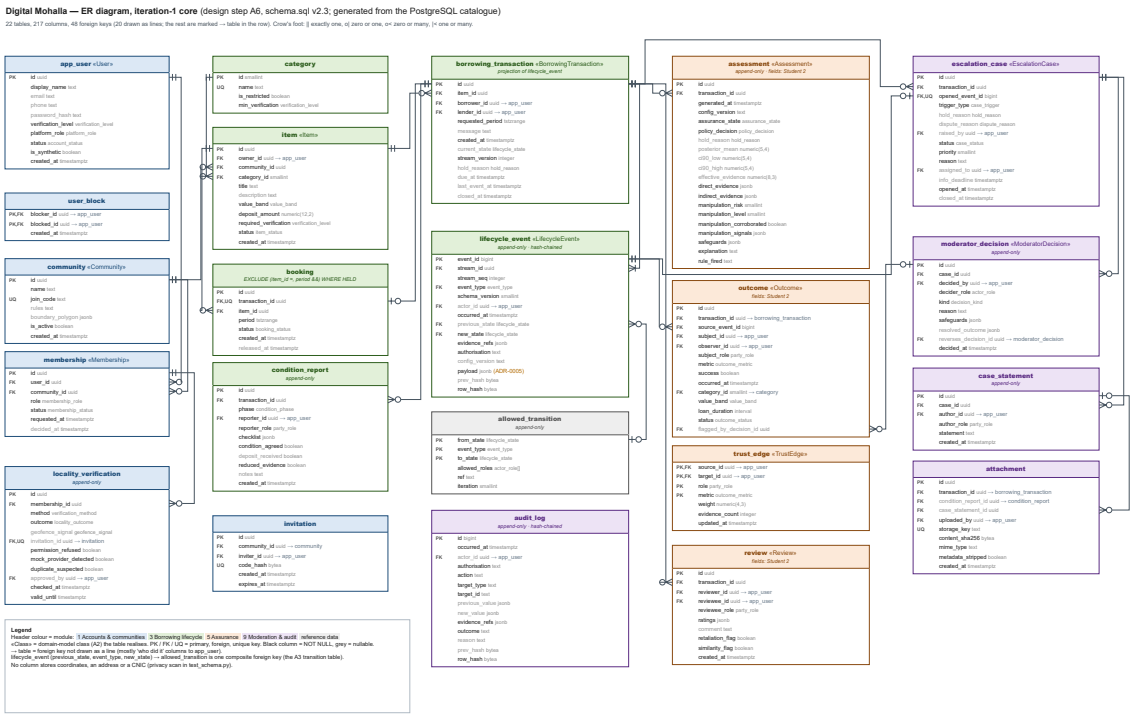


Figure 5.4: ER diagram, schema v2.3 (crow's foot, design step A6): 22 tables, 217 columns, 48 foreign keys

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/06_data_model/er_diagram.drawio`

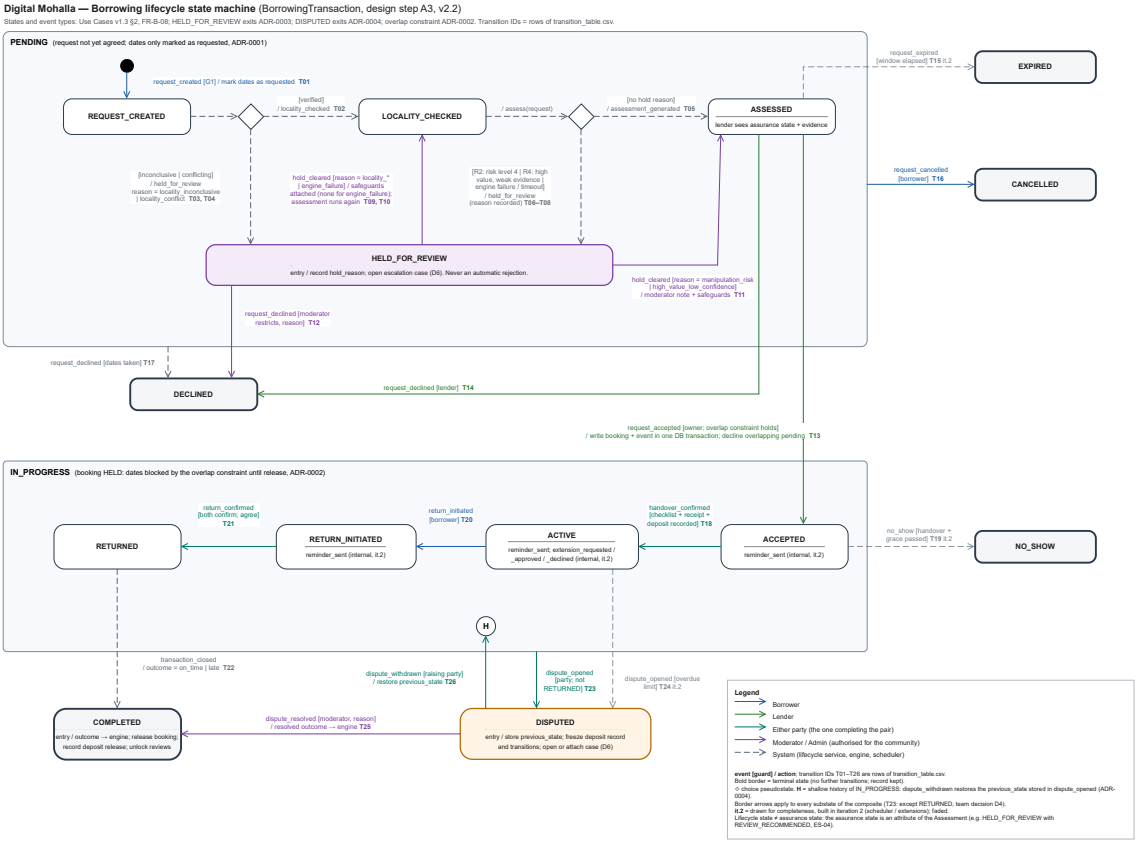


Figure 5.5: Borrowing lifecycle state machine v2.2 (UML 2.5, design step A3)

Full-size diagram: Open the full-size diagram in draw.io

Source:

`docs/design/07_state_machines/borrowing_lifecycle_state_machine.drawio`

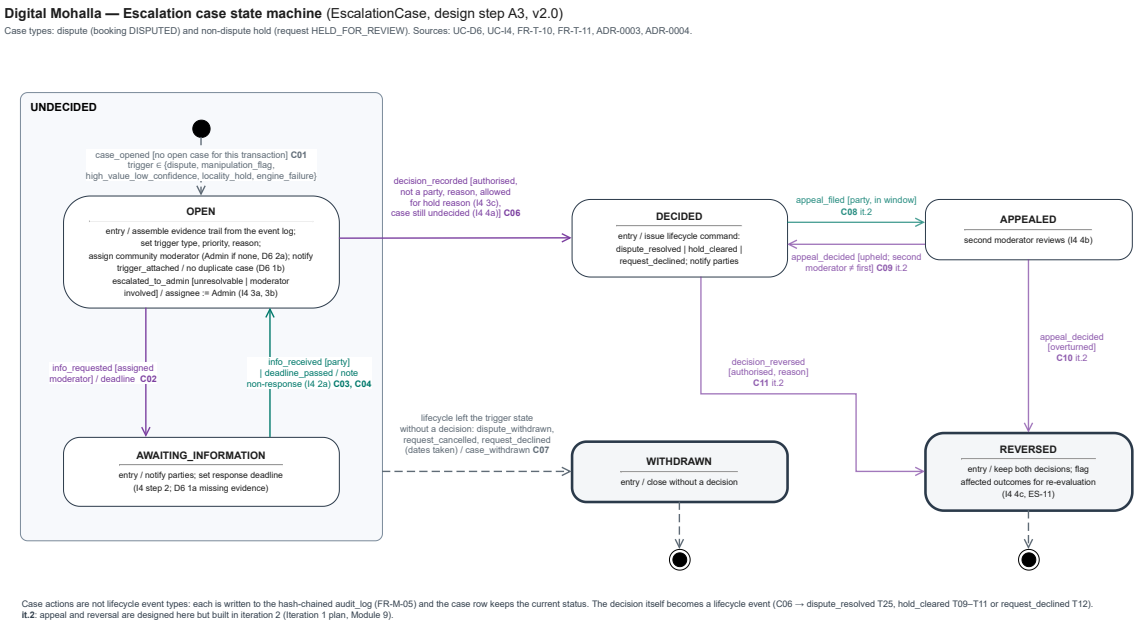


Figure 5.6: Escalation case state machine (UML 2.5, design step A3)

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/07_state_machines/escalation_case_state_machine.drawio`

5.8.3 Interactions

Reasoning. The dispute freezes the loan and opens a case in the same database transaction as the `dispute_opened` event, so there is never a disputed loan without a case. The moderator's decision is first recorded (`moderator_decision`) and then applied as one lifecycle event that references it (`evidence_refs.decision_id`), so the decision and its effect cannot diverge. Sequence (b), handover to evidence, is in Appendix E. *Serves:* FR-B-16, FR-T-10, FR-T-11. *Owner:* S3.

5.8.4 Data flow and privacy boundary

Reasoning. The context diagram shows what enters and leaves the assurance pipeline: lifecycle events and reviews in; assessments, explanations and cases out. Exact location never crosses into the pipeline: GPS is read only at membership time and reduced to a coarse result, and the request carries no location (D13). *Serves:* NFR-PR-01, NFR-PR-02. *Owner:* S2.

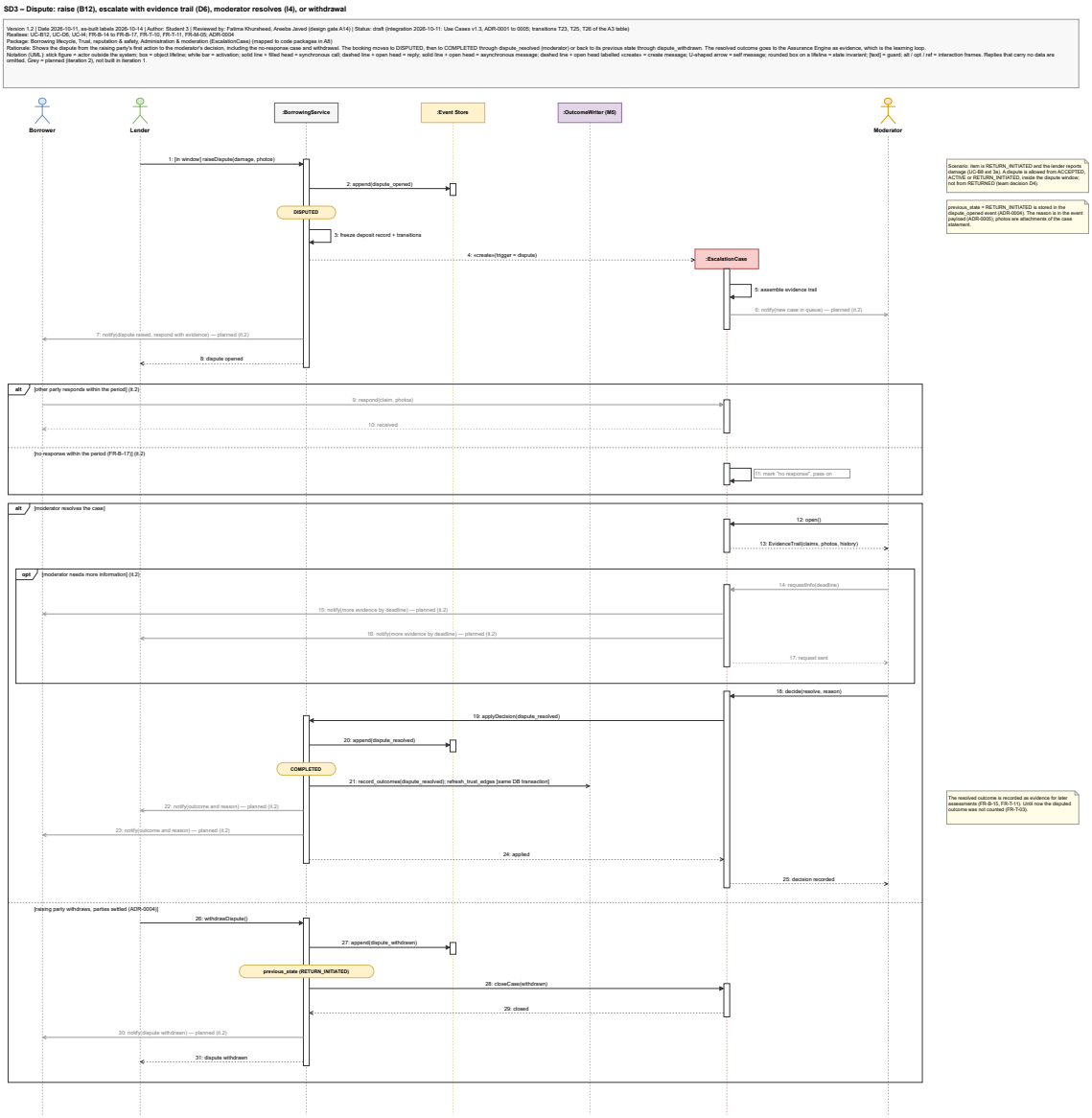


Figure 5.7: Sequence (c): raise dispute, escalate, moderator decision (UC-B12, D6, I4)

Full-size diagram: Open the full-size diagram in draw.io

Source: `docs/design/08_sequence_diagrams/sequence_c_dispute.drawio`

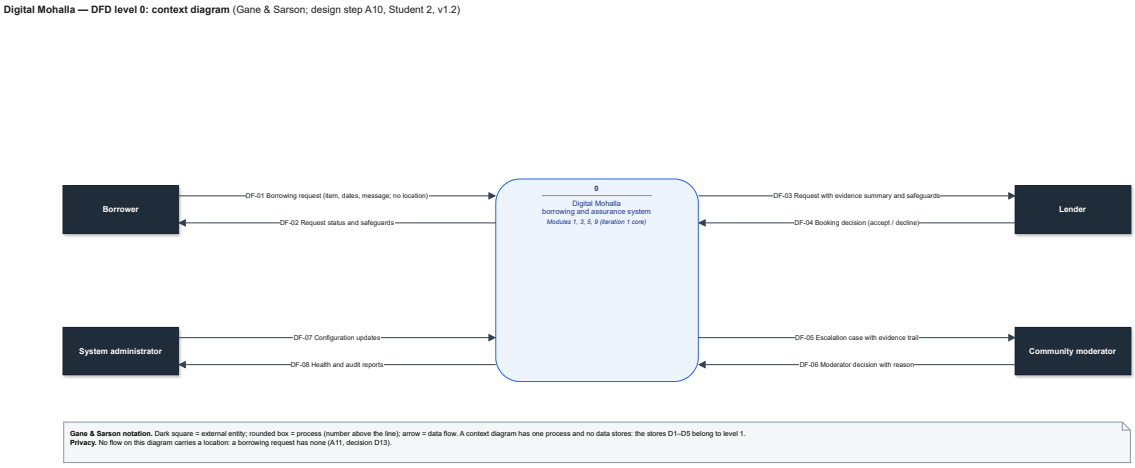


Figure 5.8: Assurance pipeline, DFD level 0 (context diagram; Gane and Sarson, design step A10)

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/10_dfd/dfd_assurance_pipeline_level0_level1.drawio`

5.8.5 Deployment

Reasoning and as-built deployment. Figure 5.9 shows the deployment as built; the provisional Part A design (one AWS host) is kept in Appendix E (Figure E.5). The Part A design fixed what any target must provide: TLS termination, one API process, PostgreSQL 16 with `btree_gist`, migrations before start-up, a `/health` check and secrets outside the repository; it marked the host itself "to be confirmed". The confirmed target keeps every one of these properties on managed services. The API runs as a Docker container on Render (`digitalmohalla-api`) that applies `alembic upgrade head` and then starts Uvicorn; the web app is an Expo web export served by Render as a static site (`digitalmohalla-web`); the database is Neon PostgreSQL 16. Render deploys `main` only after the CI checks pass, and secrets live in Render's environment settings. *Serves:* NFR-D-03, NFR-D-04, NFR-S-01.

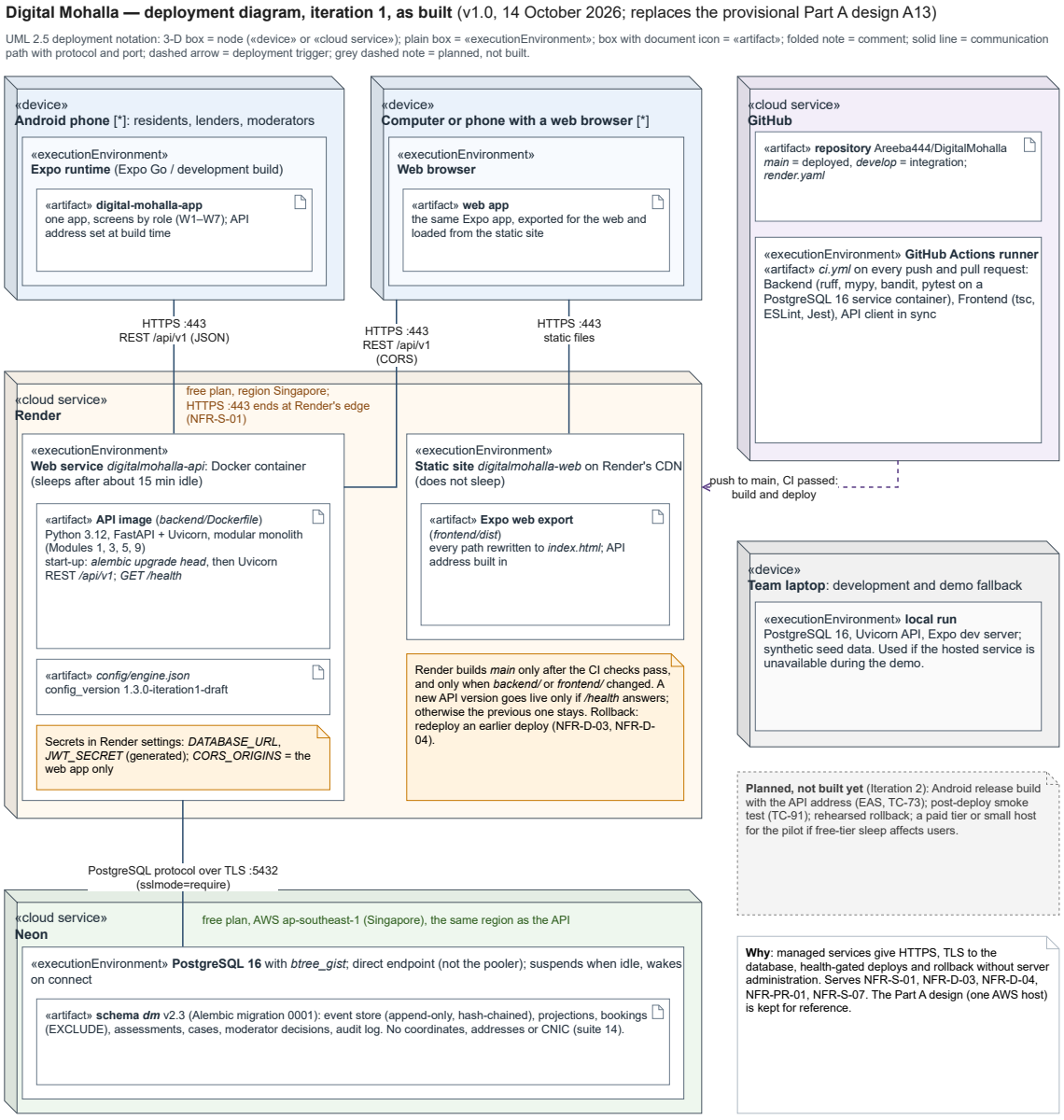


Figure 5.9: Deployment diagram, Iteration 1 as built (UML 2.5): API and web app on Render, PostgreSQL 16 on Neon, deployed from `main` after CI passes

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/04_deployment/deployment_diagram.drawio`

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

1. A live borrowing lifecycle in which invalid transitions and double bookings cannot be stored, and every state can be rebuilt from events.
2. The assurance engine on real evidence: outcomes from closed loans, the recency-weighted posterior, indirect evidence, manipulation signals and the decision policy, with an explanation on every result.
3. The human loop: disputes, holds and moderator decisions, with a resolved dispute feeding back into evidence.
4. Evidence that the engine behaves as designed: the mid-term scenarios in CI, and k-hop latency and coverage measured.
5. A pipeline in which every change is checked before merge and `main` is deployed automatically.

6.2 Selected Use Cases / Features

UC-B5 (request), B6 (respond), B7 (handover), B8 (return and complete), B12 (dispute), D3/4/5 (assess), D6 (escalate), I4 (resolve) and the log part of I6; minimal login (part of A2). These are the use cases on the path from a request to evidence and back, which is the shortest path that exercises the whole computing problem. Membership and locality (A3, A4) were designed but left for Iteration 2, because the lifecycle only needs the stored locality result.

6.3 Design

The iteration followed the Part A baseline in Chapter 5: the transition table from the state machine, the schema from the ER design, the four algorithms from the algorithm and data-structure design, and the screens from wireframes W1–W7 (Appendix E). Design questions that came up while building were settled as team decisions D10–D20 (Table 5.4) before code was merged.

6.4 Implementation

Work was organised as three vertical slices; each ended with a release pull request from `develop` to `main`, which triggered a deployment.

Table 6.1: Slices and their pull requests

Slice	Content	Pull requests
B0	Design baseline in the repository; synthetic dataset, validator and loader; CI	#1, #6, #7
A	Event store, transition table, migration; engine configuration and stub engine; login and REST API; resident web app	#2, #3, #4, #5; release #8
CD	API on Render with Neon PostgreSQL; web app as a Render static site	#17, #20; releases #18, #24
B	Decisions D10–D16 and test plan v1.3; outcome extraction (B1); Beta posterior and similarity (B2); trust edges (B2); k-hop and k sweep (B3); decision policy, explanation and real engine (B4); scenario suite (B5); lender evidence screen (B6); two-theme mobile UI	#15, #16, #19, #21, #22, #23, #25, #28, #27; releases #29, #30
C	Manipulation checks with retaliation handling (C1); disputes (C1); moderator decisions (C2); privacy checks and fuzzing (C2); moderator app and dispute screens (C3)	#31–#35; release #36

6.5 Completed Features / Module

Table 6.2: Completed features

Feature	What works	Module, owner
Event store and lifecycle	14 states, 21 event types; append-only, hash-chained events; replay in Python and SQL; invalid transitions refused twice and audited	3, S1
Request to completion	Request, locality read, assessment, accept (overlapping requests declined), decline, cancel, handover checklist and confirmation with deposit record, return, on-time or late close	3, S1
Double-booking guard	Exclusion constraint; exactly one of N simultaneous acceptances commits	3, S1
Outcome extraction	Confirmed outcomes per role and metric on close; disputed outcomes excluded until resolved	5, S2
Beta posterior	Recency and similarity weights; 90% interval; effective evidence	5, S2
Trust edges and k-hop	Edge weights from verified outcomes; bounded search with decay, path diversity, noisy-OR and cap; paths shown to the lender	5, S1
Manipulation checks	S1 partner clustering, S4 review similarity, corroboration rule, retaliation set-aside	5, S2
Decision policy and explanation	Rules R1–R9, safeguards, plain-language explanation; engine timeout or failure holds	5, S2
Disputes	Raise (freezes the loan, opens a case), view, withdraw (restores the previous state)	3/9, S3
Moderator decisions	Queue of disputes and holds for the moderator's community; decisions with a mandatory reason; resolved outcome becomes evidence	9, S3
Privacy checks	Response-model scan, whole-loan key and value checks, log and audit scan, strict request bodies, fuzzing	cross-cutting, S1
Apps	Resident app (browse, request, lender evidence, handover, return, dispute) and moderator app (queue, case adjudication), two themes	apps, S3
Synthetic data and scenarios	Seeded generator (mini, standard, scale tiers), validator, loader; scenario suite in CI	10, S3
CI/CD	Three CI jobs on every push and pull request; automatic deploy of `main` after CI	S1 (CI), Render deploys

6.6 Technical Implementation Details

6.6.1 Code organisation

The backend follows the layout in the tech-stack document: `app/api` (routes, schemas, dependencies), `app/domain` (`lifecycle`, `assurance/{estimation, indirect, detection, policy}`, `moderation`, `accounts`), `app/infra` (`db`, `eventstore`) and `app/config`. The domain packages contain no input/output. The backend application is 4,383 lines of Python in 33 files, and its tests are 5,092 lines; the frontend is about 4,200 lines of TypeScript excluding the generated client.

Table 6.3: Key files, classes and functions

File	Main elements
`domain/lifecycle/service.py`	`BorrowingService`: `create_request`, `accept`, `decline_overlapping`, `confirm_handover`, `confirm_return`, `raise_dispute`, `withdraw_dispute`, `decide`, `moderation_queue`; engine call with timeout
`domain/lifecycle/transition_table.py`	`TransitionTable.load`, `lookup`, `is_valid`, `next_state` (from `transition_table.csv`)
`domain/lifecycle/aggregate.py`	`BorrowingTransaction.apply`, `replay` (gap, order and previous-state checks)
`domain/lifecycle/ports.py`	`IAssess`, `LifecycleStore`, `AuditLog`; `AssessmentRequest`, `AssessmentResult`
`infra/eventstore/store.py`, `postgres.py`	`create_request`, `append`, `load_stream`, `replay_state`, `verify_chain`; database error mapping; `PostgresAuditLog`
`domain/assurance/engine.py`	`AssuranceEngine.assess`; `EvidenceSource` protocol
`assurance/estimation/posterior.py`, `similarity.py`, `outcomes.py`	`recency_weight`, `credible_interval`, `estimate`; similarity (D17); outcome rules (D11)
`assurance/indirect/khop.py`, `edges.py`	`find_paths`, `path_strength`, `indirect_trust`; `edge_weight`, `build_edges`
`assurance/detection/manipulation.py`	`partner_clustering`, `review_similarity`, `max_cosine`, retaliation flags, `combine`
`assurance/policy/decision.py`, `explanation.py`	`RULES`, `decide`, `safeguards_for`; `build_explanation`

File	Main elements
`domain/moderation/decisions.py`	`plan` (which event a decision produces), `InvalidDecision`
`api/routes.py`, `schemas.py`	21 API endpoints (plus `/health` in `main.py`); `StrictIn` request base (unknown fields and NUL characters refused)
`config/engine.json`	Versioned parameters (`1.3.0-iteration1-draft`), one section per algorithm
`frontend/src/app/(resident)`, `(moderator)`	Expo Router screens; `src/api` generated typed client; `AssurancePanel`

6.6.2 Configuration

Every tunable value lives in `backend/config/engine.json`, marked `initial_hypothesis_unvalidated`, with a sensitivity range for the FYP-2 study. The loader refuses a file with a missing or out-of-range parameter (TC-74), and every stored assessment records the version it used. The version has moved from `1.0.0` (initial parameter set) and `1.1.0` (integration: S4 active, R6_SPARSE, ADR-0007 and ADR-0008), both in Part A, to `1.2.0` (similarity, D17) and `1.3.0` (manipulation checks, D18, D19), each change recorded in the file's changelog.

6.6.3 Coding conventions and static analysis

Python follows ruff's lint rules and formatter (line length 100); the domain packages pass `mypy`; `bandit` scans for insecure patterns and `pip-audit` for vulnerable dependencies. TypeScript is checked with `tsc` and ESLint, and formatted with Prettier. Every module that contains maths or a non-obvious rule starts with a plain-language note explaining what it does and why it was chosen over the alternative, so that any member can explain it.

6.7 Algorithms and Technical Decisions

The four algorithms below are implemented as specified in the Part A algorithm and data-structure design. All parameter values quoted are those of the current configuration, version `1.3.0-iteration1-draft`.

6.7.1 Algorithm 1: recency-weighted Beta posterior (S2)

Each confirmed outcome i of the borrower in the borrower role is a success ($y_i = 1$) or a failure ($y_i = 0$) for one metric. An outcome d_i days old with similarity σ_i to the request has weight

$$w_i = 2^{-d_i/h} \max(\sigma_{\min}, \sigma_i), \qquad h = 180 \text{ days}, \ \sigma_{\min} = 0.5,$$

where $\sigma_i = 0.4\, c_{\text{band}} + 0.4\,[\text{same category}] + 0.2\,\frac{\text{shorter duration}}{\text{longer duration}}$ and c_{band} is 1, 0.5 or 0 for the same, an adjacent or a distant value band (D17). With $s = \sum w_i y_i$ and $f = \sum w_i (1 - y_i)$ the posterior is Beta$(1+s,\, 1+f)$; the engine reports its mean, the 5% and 95% quantiles (the 90% credible interval) and the effective evidence $n_{\text{eff}} = s + f$. With no history this is Beta$(1,1)$: mean 0.5, interval $[0.05, 0.95]$, which the policy reports as INSUFFICIENT_EVIDENCE, not as a low value. *Check values tested*: weights 0.962 and 0.25 at 10 and 360 days; 18 successes and 2 failures give Beta$(19,3)$. *Complexity*: $O(n)$ time for n outcomes, $O(1)$ extra space. *Why*: a conjugate posterior gives an honest interval from very few observations [3], and the half-life lets old behaviour fade without being forgotten.

6.7.2 Algorithm 2: bounded k-hop indirect evidence (S1)

Trust edges are derived only from verified outcomes (ADR-0006): the weight of edge (a, b) is the posterior mean of a's verified outcomes with b. For a lender u and borrower v:

1. Find simple paths from u to v of at most $k = 3$ edges (depth-first, no repeated node, so cycles are safe).
2. Score a path P with h edges as $q(P) = \gamma^{h-1} \prod_{e \in P} w_e$, with $\gamma = 0.7$ (the first hop is not penalised).
3. Keep only the strongest path per first-hop neighbour; paths through the same neighbour are not independent evidence.
4. Combine with noisy-OR: $I = 1 - \prod_P (1 - q(P))$.
5. Reliability $\rho = \min\big(0.6,\ \min(1, |P|/3) \cdot \min(1, |F|/2) \cdot I\big)$, where $|F|$ is the number of distinct first hops.
6. Indirect estimate $t_{\text{ind}} = b + \rho\,(t_v - b)$, with population baseline $b = 0.5$.

The cap $\rho \le 0.6$ keeps indirect evidence below direct evidence, and the policy never lets ρ alone produce SUFFICIENT_EVIDENCE. *Complexity*: $O\big(\sum_{i=1}^{k} \Delta^i\big)$ for maximum out-degree Δ. On the hand-worked 8-node graph the code reproduces the designed values ($I = 0.7164$, $\rho = 0.6$ capped; $\rho = 0, 0.424, 0.600, 0.600$ for $k = 1..4$). *Why*: trust that weakens

with distance follows Golbeck [6]; the cap, the path-diversity rule and the bounded k are the project's adaptations for small, dense neighbourhood graphs.

k sweep. Table 6.4 reports latency and coverage for $k = 1$ to 4 on the synthetic *scale* tier (2,026 users, 5,987 edges, seed 20261001), one search per (lender, borrower) pair of 2,000 loans. $k = 3$ finds a path for 93.2% of pairs at 4.5 ms (P95); $k = 4$ adds one percentage point of coverage at 3.5 times the P95 latency and 7 times the maximum, so $k = 3$ is kept. The sweep was measured with configuration 1.2.0; version 1.3.0 changed only manipulation-detection parameters, so the k-hop results still apply. *Caveat:* the pairs are existing loans, so a direct edge between lender and borrower usually exists already, and the $k = 1$ coverage (80.3%) includes the assessed loan's own edge. This overstates coverage for genuinely new pairs; the FYP-2 evaluation uses time-ordered held-out pairs instead.

Table 6.4: k sweep on the scale tier (SYNTHETIC data, config 1.2.0)

k	P50 (ms)	P95 (ms)	max (ms)	coverage	pairs, $\rho \geq \rho_{\text{sup}}$
1	0.025	0.080	3.871	80.3%	0
2	0.087	0.709	15.543	88.9%	946
3	0.439	4.519	28.488	93.2%	1,346
4	2.279	15.915	207.045	94.2%	1,435

Windows AMD64, Python 3.12.11, commit `f7afd92`; raw output in `docs/design/14_testing/results/`.

6.7.3 Algorithm 3: manipulation-risk scoring (S2)

Manipulation risk is a separate 0–100 dimension, never averaged with trust. Iteration 1 computes two signals and one check over a 90-day window:

- **S1, partner clustering (D18).** Take the borrower's review partners (at least 3). Let r be the share of pairs of partners who reviewed each other in both directions; $S_1 = \text{clip}\big((r - 0.3)/0.6\big)$. A ring of accounts fills almost every pair; an ordinary borrower's lenders rarely review one another. The simpler "A rated B and B rated A" rule was rejected because every loan produces two reviews: on the standard tier it flagged 6 of 8 ordinary members; partner clustering flags 0 of 9.
- **S4, similar review texts.** The reviews the borrower received, as character 3-grams weighted by TF-IDF; the highest cosine similarity c between two of them gives $S_4 = \text{clip}\big((c - 0.7)/0.25\big)$.
- **Retaliation check (D19).** A low review (lowest rating ≤ 2) posted within 14 days after the subject complained about its author, about a loan whose return was con-

firmed, is set aside: left out of S1 and S4, reported, and explained to the lender in neutral words.

The score is the weighted mean over the available signals, score $= 100 \cdot \sum_i w_i s_i / \sum_i w_i$ (with $w_{S1} = 0.3$, $w_{S4} = 0.2$; a signal without enough data is left out of both sums), banded into levels 1 to 4 at 20, 40 and 60. Level 4 additionally requires two signals ≥ 0.5 (ADR-0007); otherwise the result is level 3. A legitimate family group therefore stays at level 3 (TC-38): extra verification, no hold. *Complexity*: S1 $O(p^2)$ for p partners; S4 $O(m^2 L)$ for m reviews of length L. *Why*: collusion shows as groups that rate one another [9, 10], while two-sided reviews are reciprocal by nature [11, 12]. *Known limit*: a ring that also trades with outsiders lowers its own density (camouflage).

6.7.4 Algorithm 4: decision policy (S2)

The first rule whose condition holds decides (Table 6.5). Thresholds (D12): $\tau_\mu = 0.75$, $\tau_L = 0.60$, $n_{\min} = 3$, $\rho_{\text{sup}} = 0.3$.

Table 6.5: Decision rules in order (Algorithm 4)

Rule	Condition	State	Lifecycle
R1	An open dispute involves the borrower	ACTIVE_DISPUTE	ASSESSED
R2	Manipulation level 4 (corroborated)	REVIEW_RECOMMENDED	HELD (`manipulation_risk`)
R3	No verified history and $\rho < \rho_{\text{sup}}$	INSUFFICIENT_EVIDENCE	ASSESSED
R4	High value and (similarity < 0.5 or lower bound $< \tau_L$)	REVIEW_RECOMMENDED	HELD (`high_value_low_confidence`)
R5	Manipulation level 3	REVIEW_RECOMMENDED	ASSESSED
R6_DISPUTE	A resolved dispute in the last 12 months	REVIEW_RECOMMENDED	ASSESSED
R6_SPARSE	$n_{\text{eff}} < n_{\min}$ and $\rho < \rho_{\text{sup}}$	INSUFFICIENT_EVIDENCE	ASSESSED
R7	Mean $\geq \tau_\mu$, lower bound $\geq \tau_L$, $n_{\text{eff}} \geq n_{\min}$, level 1, standard value, verified	SUFFICIENT_EVIDENCE	ASSESSED

Rule	Condition	State	Lifecycle
R8	As R7 but level 2, or medium or high value	SUFFICIENT_ EVIDENCE with safeguards	ASSESSED
R9	Anything else (e.g. sparse but supported by community paths)	REVIEW_ RECOMMENDED	ASSESSED

Because R7 and R8 need $n_{\text{eff}} \geq 3$ from the borrower's own outcomes, indirect evidence alone can never yield SUFFICIENT_EVIDENCE (property-tested, TC-34). Each rule maps to a policy decision and a set of safeguards (deposit record, handover photos and checklist, fixed return date, verification), and the explanation names the rule, the interval, the effective evidence, the community paths and the signals. The database refuses an assessment row whose rule, state, decision and hold reason disagree (`assessment_rule_ck`). *Complexity*: $O(1)$ given the inputs; a whole assessment is $O(n) + O(\Delta^k) + O(p^2 + m^2 L)$.

6.7.5 Other technical decisions

- **Engine timeout.** The lifecycle calls the engine in a worker with a 2,000 ms timeout from the configuration; a timeout or exception holds the request with `engine_failure` and stores no assurance state.
- **Moderator decisions.** `plan()` turns a decision into exactly one lifecycle event (`dispute_resolved`, `hold_cleared` or `request_declined`) according to the hold reason, and refuses invalid combinations, such as safeguards on an engine-failure hold.
- **Strict request bodies.** Every request schema refuses unknown fields and NUL characters, so a client cannot inject `actor_id`, `authorisation` or `new_state`.

6.8 Tools, Frameworks and Technologies

Table 6.6: Technology stack

Layer	Choice
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2 (synchronous) with psycopg 3, Alembic, NumPy and SciPy (Beta quantiles), argon2 password hashing, JWT tokens, managed with uv
Database	PostgreSQL 16 with `btree_gist`; triggers and constraints for the event store
Frontend	React Native with Expo SDK 57, Expo Router, React Native Paper, TanStack Query, `openapi-fetch` with a client generated by `openapi-typescript`; Android and web
Testing	pytest, Hypothesis (property-based tests and fuzzing), Jest with React Native Testing Library
Quality	ruff, mypy, bandit, pip-audit, ESLint, tsc, Prettier, npm audit, Dependabot
CI/CD	GitHub Actions; Render (Docker web service and static site); Neon PostgreSQL

6.9 Demonstration Evidence

- **Live system.** API: `https://digitalmohalla-api.onrender.com` (`/health` returns 200); web app: `https://digitalmohalla-web.onrender.com`. Both returned HTTP 200 on 14 October 2026. Demo accounts and data are synthetic.
- **Repository.** `https://github.com/Areeba444/DigitalMohalla`: 36 pull requests (30 merged), CI history, release pull requests to `main`.
- **Screens.** The built screens follow wireframes W1–W7 (Appendix E): the lender's request view with the assurance panel (W1), decision (W2), handover (W3), return (W4), dispute form (W5), moderator queue (W6) and case adjudication (W7).
- **Demo script.** (1) A newcomer's request gives INSUFFICIENT_EVIDENCE with a wide interval and safeguards; (2) a borrower known only through neighbours (ES-13) shows the community paths; (3) an established borrower gives SUFFICIENT_EVIDENCE; (4) accept, hand over, return with damage, dispute; (5) the next request shows ACTIVE_DISPUTE; (6) the moderator decides with a reason and the

loan completes; plus an invalid transition rejected, two simultaneous acceptances, the colluding scenario held for review, and the CI pipeline.

6.10 Iteration Outcome

Iteration 1 met its objectives: all three slices are merged and deployed, the mid-term scenario target is met (7 of 7), and 623 backend and 29 frontend tests pass. What is left from the plan is listed in Section 3.8; none of it blocks the demo story.

Chapter 7

Testing and Early Validation

Testing follows *Test Plan v1.3* (`docs/design/14_testing/Test_Plan.md`): 19 suites, test cases written as input / expected / actual, and traceability to requirements. All test data is synthetic.

7.1 Testing Strategy

- **Test the invariants where they are enforced.** Rules that must never break (valid transitions, no overlapping bookings, append-only events, `config_version` on every assessment) are enforced in PostgreSQL, so they are tested against a real PostgreSQL 16 database, not a mock.
- **Test the maths against worked numbers.** Every algorithm was worked by hand in Part A (an 8-node graph, thirteen scenario fixtures, check values such as Beta(19,3)); the code must reproduce those numbers. The *k*-hop search is also cross-checked against NetworkX.
- **Test properties, not only examples.** Hypothesis generates inputs for the posterior (the mean always lies inside its interval; another success never lowers the mean), the policy (indirect evidence alone is never SUFFICIENT), the *k*-hop search and the API (fuzzing).
- **Test behaviour end to end.** The scenario suite drives the real engine through the real lifecycle on a synthetic neighbourhood and checks the state, the rule and the explanation.
- **Deterministic.** Fixed seeds (dataset seed 20261001), no network calls, and the same tests in CI as on a laptop.
- **Gate every change.** A pull request cannot be merged until the three CI jobs pass (NFR-D-04).

7.2 Test Levels and Techniques

Table 7.1: Test levels

Level	Tests	What is tested
Unit (backend)	477	Transition table (all 294 pairs), aggregate and replay, posterior, similarity, outcome rules, trust edges, k-hop, manipulation signals, decision policy, explanation, moderation rules, configuration, generator
Integration (backend)	102	Event store and triggers, lifecycle service, concurrency, authorisation, disputes, moderation, outcome and trust-edge feeds, privacy, API, fuzzing, engine on the synthetic dataset
Scenario / acceptance	44	ES scenarios on the fixtures and end to end; lender evidence API
Frontend component	29	Screens W1, W5, W6 and W7 rendered with React Native Testing Library against a mocked API; formatting and moderation helper functions
Total	652	623 backend, 29 frontend

Techniques: state-transition testing (the full matrix), decision-table testing (rules R1–R9, moderator decisions), boundary values (value bands at 5,000/5,001 and 15,000/15,001 PKR; 10 and 360 days of age), concurrency testing (parallel acceptances behind a barrier), fault injection (engine timeout and exception), property-based testing, oracle testing (worked examples, NetworkX), fuzzing (API bodies), and static analysis (ruff, mypy, bandit, ESLint, tsc).

7.3 Test Cases

Table 7.2 shows representative test cases, one or more per suite; Appendix C lists all implemented ones.

Table 7.2: Representative test cases (input / expected / actual)

TC	Input	Expected	Actual
TC-01	Each of the 294 state–event pairs	31 valid pairs accepted; 263 refused	Pass (297 tests)
TC-02	COMPLETED loan; lender sends accept	409 “invalid transition”; audit entry	Pass
TC-05	RETURNED loan; dispute	Refused (decision D4)	Pass
TC-07	Happy path, 8 events	Replay equals the stored state	Pass
TC-08	5 parallel acceptances, overlapping dates, 3 rounds	Exactly one ACCEPTED per round; others DECLINED “dates taken”	Pass
TC-13/14	UPDATE or DELETE on events; an edited event	Refused; tampering detected by `verify_event_chain()`	Pass
TC-20	No outcomes	Beta(1,1), mean 0.5, 90% CI [0.05, 0.95]	Pass
TC-22	Ages 10 and 360 days, $h = 180$	Weights 0.962 and 0.25	Pass
TC-25	8-node hand-worked graph, $k = 3$	3 paths (q = 0.54, 0.2195, 0.21), $I = 0.7164$, $\rho = 0.6$	Pass
TC-28	Graph with a cycle and a self-loop	Search terminates; no repeated node	Pass
TC-34	Hypothesis: only indirect evidence is strong	Never SUFFICIENT_ EVIDENCE	Pass
TC-36	Engine stub sleeps past the timeout	HELD_FOR_REVIEW (`engine_failure`); no assessment	Pass
TC-38	Family group, S1 = 1.0, S4 low	Level 3, rule R5, not held	Pass
TC-43	ES-04 colluding ring	R2; HELD_FOR_REVIEW (`manipulation_risk`); case opened; “not proof” wording	Pass

TC	Input	Expected	Actual
TC-46	ES-13 community paths	R9, REVIEW_RECOMMENDED; both paths listed	Pass
TC-50	ACTIVE loan; lender disputes	DISPUTED; previous state stored; case opened	Pass
TC-53	Moderator resolves with a reason	COMPLETED via `dispute_resolved`; outcome recorded	Pass
TC-55	Engine-failure hold; safeguards imposed	Refused	Pass
TC-58	Every API response model	No address, coordinate, CNIC, phone or e-mail field	Pass
TC-84	Fuzzed bodies (injection strings, malformed values)	422 for bad input, never 500	Pass
TC-100	W1 with ES-01, ES-02, ES-13 assessments	State as text, interval, evidence, no single number	Pass
TC-106	W6 moderator queue	Own community only; disputes and holds together	Pass

7.4 Test Results

CI evidence. GitHub Actions run 38464197964, on the push of commit `b47243a` to `main` (13 October 2026, 20:36 UTC), passed all three jobs: the backend suite ran **623 tests, all passed, in 30.4 s** on Linux with a PostgreSQL 16 service container, with **97% line coverage** of the backend application code (1,850 statements, 49 not executed); the frontend suite ran **29 tests in 4 suites, all passed**; static checks (ruff, ruff format, mypy, bandit) and the API-client check passed.

Local re-run. The same backend suite was re-run on 14 October on a Windows laptop with PostgreSQL 16: 623 passed in 13 min 33 s (the difference is the slower machine and the fuzz tests, Table 7.5, DF-07). The breakdown below is from that run.

Table 7.3: Backend results by area (14 October 2026)

Area (test files)	Tests passed
Lifecycle: transition table, aggregate, actions	311
Event store, lifecycle service, disputes	27
Assurance maths: posterior and properties, similarity, outcome rules, trust edges, *k*-hop	69
Detection and policy: manipulation, decision policy, explanation, engine, configuration	66
Moderation: rules and service	28
Data feeds on PostgreSQL: outcomes, trust edges, dataset load, engine and *k*-hop on dataset	34
API, evidence endpoint, privacy, fuzzing	23
Scenarios (fixtures, end to end, lender API)	44
Generator and health	10
Rule checks in the database	11
Total	623

7.4.1 Scenario results (NFR-Q-01)

A scenario passes only if both the state and the explanation match. Fixture tests feed each scenario's hand-checked numbers through the real policy and explanation builder; end-to-end tests send a request on the synthetic *mini* dataset through the lifecycle, the real engine and PostgreSQL.

Table 7.4: Evaluation scenarios after Iteration 1 (SYNTHETIC data)

ES	Scenario	Expected state (rule)	Fixture	End to end	Required at mid
01	New user, no history	INSUFFICIENT (R3)	Pass	Pass	Yes
02	Consistently reliable	SUFFICIENT (R7)	Pass	Pass	Yes

ES	Scenario	Expected state (rule)	Fixture	End to end	Required at mid
03	Reliable, one recent resolved dispute	REVIEW (R6_ DISPUTE); ACTIVE_DISPUTE (R1) while open	Pass	Pass	Yes
04	Colluding ring	REVIEW (R2), held	Pass	Pass	Yes
05	Retaliatory review	SUFFICIENT with safeguards (R8)	Pass	Pass	Yes
06	High value, weak evidence	REVIEW (R4), held	Pass	Pass	Yes
13	Known only through neighbours	REVIEW (R9); INSUFFICIENT (R6_SPARSE) without paths	Pass	Pass	Yes
07	Conflicting condition reports	ACTIVE_DISPUTE (R1)	–	State	No
08	Failure to return	ACTIVE_DISPUTE (R1)	–	State	No
11	Moderator reversal	REVIEW	–	State	No
12	Sparse or old history	INSUFFICIENT	–	State	No
09	Location mismatch	REVIEW	–	–	No (locality, It. 2)
10	Location permission refused	INSUFFICIENT	–	–	No (locality, It. 2)

Result: 7 of the 7 scenarios required at mid-term pass with state and explanation (target: at least 7 of 13). A further four (ES-07, 08, 11, 12) already return their expected state end to end; their full workflows (overdue escalation, reversal) are Iteration 2 work. ES-09 and ES-10 need the locality method.

7.5 Acceptance Testing / Validation

Validation in Iteration 1 is against the acceptance criteria of FR/NFR Part E and the scenario target of NFR-Q-01: each criterion in Table 4.4 has a passing test. Validation with people has not started: the internal alpha session (the team running the demo story on the deployed instance, Test Plan §12.1) is scheduled before the demo, and user acceptance with residents (SUS, task completion, understanding of the explanation) belongs to the FYP-2 pilot. The engine's *quality* (whether its uncertainty improves decisions over averages) is not claimed here; it is the subject of the FYP-2 evaluation (Chapter 9).

7.6 Edge and Failure Cases

- **Concurrency:** simultaneous acceptances for overlapping dates; a stale command (409, nothing appended).
- **Invalid input and transitions:** every invalid state–event pair; disputes from RETURNED; a moderator clearing a manipulation hold to the wrong state; safeguards on an engine-failure hold; a decision without a reason.
- **Authorisation:** a non-owner accepting; a non-party disputing; the other party withdrawing a dispute; a moderator from another community; a party deciding its own case.
- **Engine failure:** timeout and exception both hold with `engine_failure`.
- **Tampering:** UPDATE/DELETE refused; an edited event detected.
- **Graph edge cases:** cycles, self-loops, no path, paths longer than k.
- **Hostile input:** unknown fields (e.g. latitude, `new_state`) refused with 422; NUL characters and injection strings refused; no database error text leaked.

7.7 Defects and Corrective Actions

Table 7.5: Defect log (Iteration 1)

ID	Defect	Found by	Correction
DF-01	The written S1 formula (pairwise reciprocity) flagged 6 of 8 ordinary members, and ES-05's borrower scored like the ES-04 ring	Measurement on the synthetic tiers	S1 redefined as partner clustering (D18); 0 of 9 ordinary members flagged
DF-02	A NUL character in any text field reached PostgreSQL and caused a 500 error	Fuzzing (TC-84)	Request bodies refuse NUL characters with 422 (`d4e4dec`)
DF-03	Unknown fields were silently ignored, so a client could send a latitude or an address without an error	Privacy test TC-60	All request bodies refuse unknown fields (`StrictIn`)
DF-04	Dispute and decision bodies added in parallel branches were not strict	Guard test, after merging `develop`	Made strict (`6856b6d`)
DF-05	ES-02 and ES-05 data did not carry the evidence their rules need once similarity weighting was added (n_{eff} 4.1 and 2.0)	Scenario suite	Synthetic casts rebuilt; both pass (`5bd6a8c`)
DF-06	The fuzz test generated an infinite float that JSON cannot encode	CI failure on PR #34	Strategy excludes infinity (`275f658`)
DF-07	Fuzz tests took 28 minutes because every example logged in (argon2)	Local run	Tokens cached per user; the fuzz suite now takes about 5 minutes
DF-08	An `assert` used as a runtime check in the outcome writer (removed under `python -O`)	bandit	Explicit check (`5873d0f`)
DF-09	A Jest screen test timed out on its first render when run alongside the backend suite	Local run, 14 October	First-render timeout raised to 20 s; passes when run alone; open as a flaky-test risk

7.8 Current Validation Status

- **Verified by tests:** all Iteration 1 reliability, security and privacy gates (NFR-R-01 to R-04, S-02, S-03, S-05, S-07, PR-01, D-01, D-04) and NFR-Q-01 at mid-term.
- **Measured, not gated:** k-hop latency and coverage for $k = 1$ to 4.
- **Not yet run:** full-assessment latency (NFR-P-03) and transition commit latency under load (TC-61, NFR-P-04), both in Iteration 2; deployed-instance system tests and smoke flow (suite 15, TC-91, TC-92), the Android device test (TC-73), upload tests (deferred, D20).
- **Not yet claimed:** decision quality against baselines, calibration and fairness (FYP-2).

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

- **Design:** the Part A baseline (requirements and use cases v1.3, eight ADRs, domain model, state machines, data dictionary, ER schema v2.3, class, component, layer, sequence, activity and DFD diagrams, algorithm design, locality design, wireframes, test plan v1.3) and decisions D1–D20.
- **Implementation:** Slices A, B and C (Chapter 6), deployed on Render and Neon.
- **Verification:** 623 backend and 29 frontend tests; the mid-term scenario target met; the k sweep measured.
- **Process:** 30 merged pull requests, six release pull requests to `main`, CI on every pull request, an AI usage log and a decisions log.

8.2 Work Remaining

For FYP-1 final (Iteration 2): membership and the locality method; scheduler jobs, extensions and counter-offers; uploads; reviews and the remaining manipulation signals; information requests, appeals and the audit-log screen; in-app notifications; the simulator, baselines and the written FYP-2 evaluation plan. Before the mid-term demo: the release tag and the alpha session on the deployed instance. The latency tests (NFR-P-03, NFR-P-04) run in Iteration 2. Details in Chapter 9.

8.3 Individual Responsibilities

Responsibilities follow the Iteration 1 plan's work division (Table 3.3): S1 Fatima Khursheed owns the lifecycle, indirect trust, locality and CI; S2 Areeba Javed owns estimation, the decision policy and manipulation detection; S3 Minahil Kashif owns moderation, evaluation (scenarios and synthetic data) and the apps. Two pieces of work changed hands: deployment and the two-theme mobile UI (#27), both planned for S3 and carried out by S1.

8.4 Contribution of Each Team Member

Fatima Khursheed (S1). Part A: ADRs, lifecycle state machines and transition table, data dictionary, ER schema, architecture (component and layer diagrams), locality design (A11), the indirect-trust part of the algorithm design, sequence (b), and the integration of the three members' artefacts into one baseline. Part B: lifecycle core and event store (#2), login and REST API (#4), CI (#7), trust edges (#21), k-hop search and k sweep (#22), deployment of the API and web app (#17, #20), the two-theme mobile UI (#27), and privacy checks with fuzzing (#34). 39 commits.

Areeba Javed (S2). Part A: the algorithm and data-structure design (Algorithms 1, 3 and 4), the engine configuration, the decision-policy activity diagram, sequence (a) and the DFD. Part B: versioned configuration and the Slice A engine stub (#3), outcome extraction (#16), the recency-weighted Beta posterior and the similarity formula (D17) (#19), the decision policy, explanation and real engine (#23), the manipulation checks with retaliation handling (#31), and team decisions D10–D16 (#15). 37 commits.

Minahil Kashif (S3). Part A: sequence (c), the dispute activity diagram, SSDs, the use case diagram, wireframes W1–W7 and the test plan. Part B: project scaffold, the resident app (#5), the synthetic dataset generator, validator and loader (#6), the scenario suite (#25), the lender evidence screen (#28), disputes (#32), moderator decisions (#33), the moderator app (#35), and the defence slides document (#26). 50 commits.

All three members took part in the design review, the team decisions and the review of one another's pull requests.

8.5 Repository and Version Control Evidence

- **Repository:** `https://github.com/Areeba444/DigitalMohalla`.
- **Branches:** `main` (deployed; changed only by release pull requests from `develop`),

`develop` (integration), and one branch per piece of work: `feature/...`, `docs/...`, `ci/...`. 24 feature, documentation and CI branches were merged through pull requests.

- **Reviews:** 22 of the 24 feature pull requests were reviewed by another member before merge (#1 and #15 were joint pull requests by all three members; in #7 the reviewer, Areeba, had also contributed a commit). #21 and #27 were merged without a review; Areeba reviewed and approved both after merge, on 14 October. GitHub Copilot's automatic reviewer also commented on 13 pull requests (Section 8.7).
- **CI evidence:** every pull request shows the three checks (Backend, Frontend, API client in sync); a failing check on PR #34 (DF-06) was fixed before the pull request was merged.
- **Dependabot:** six dependency-update pull requests are open and are reviewed before merge, not merged automatically.

8.6 Milestones and Commit Evidence

Table 8.1: Release milestones (merge times in Pakistan time)

Release PR	Merged	Content
#8	13 Oct, 04:35	Slice A: lifecycle loop, API, login, resident app, CI, dataset
#18	13 Oct, 05:32	Outcome extraction; API deployment
#24	13 Oct, 12:29	Assurance engine; web app deployment
#29	13 Oct, 13:44	Slice B complete: scenario suite, lender evidence screen
#30	13 Oct, 14:10	Mobile UI, two themes
#36	14 Oct, 00:27	Slice C: human loop; feature freeze

126 commits (excluding merges) are on `main`: Minahil Kashif 50, Fatima Khursheed 39, Areeba Javed 37. Each commit is authored by the member who owns the change, and 125 of the 126 carry a `Co-Authored-By: Claude` trailer marking AI assistance. The repository history starts on 12 October 2026: the design baseline was prepared before that and committed to the repository on 13 October (PR #1), and implementation took place

between 12 and 14 October. The release tag `v0.1.0-rc1` has not been created yet; it will mark the demo version.

8.7 Tools and Tool-Assisted Development

The team used **Claude Code** (Anthropic's AI coding assistant, model Claude Opus 5.5) during Iteration 1, and **GitHub Copilot** in two places (an earlier draft of this report, and automatic pull-request review comments). Use is declared per module at two levels:

- **L2, assisted:** GenAI produced code, tests, diagrams or document drafts that the team reviewed, modified, integrated and tested, and can explain.
- **L3, core-assist:** GenAI was used inside a core feature; for these modules the team states below what was generated, what the team contributed, why the design works and its limits.

Every substantial AI-assisted change has a row in `docs/ai-usage-log.md` (date, tool, work, files and reviewer), and AI-assisted commits carry a `Co-Authored-By: Claude` trailer, so the repository history itself shows which commits had AI help.

Working method. The design baseline came first. For each implementation task: (1) the module owner specified the task from the baseline (module, requirement, test cases); (2) Claude Code drafted the code and tests; (3) the team reviewed the draft line by line and asked for changes where it departed from the design; (4) the change had to pass CI (ruff, mypy, bandit, the backend and frontend test suites); (5) it was merged through a reviewed pull request. Most first-draft code in the repository was produced this way.

Table 8.2: GenAI use by module

Module (owner)	Level	GenAI produced	Team contribution	Why the design works	Known limits
Assurance engine: estimation and decision policy (S2)	L3	Python implementation of the posterior, rules R1–R9, the explanation builder; unit and scenario tests	Algorithm design (Algorithm 1 and 4); outcome mapping (D11); thresholds (D12); similarity formula (D17, decided by S2); hand-checked expected results for ES-05 and ES-13 (D13, D14)	The posterior reports its own uncertainty; zero history is INSUFFICIENT, not negative; every result names the rule that fired	Thresholds are initial hypotheses checked only on synthetic data
Indirect trust (S1)	L3	k-hop search, edge projection and the k sweep harness	Edge-weight rule (ADR-0006, D10, confirmed by S2); k-hop design (k, γ, noisy-OR, cap); the hand-worked example graph	Indirect evidence is capped and never sufficient alone (ADR-0008); $k = 3$ chosen from the measured sweep	ADR-0006 accepted as experimental
Manipulation checks (S2)	L3	S1 partner clustering, S4 review similarity, retaliation set-aside and tests	Rejected the first S1 version after measuring that it flagged ordinary members; D18 and D19 were proposed with AI help from measurements and published work and confirmed by S2	Signals only raise a review flag; a hold needs two signals and a moderator decides (ADR-0007)	S2, S3, S5 in Iteration 2; synthetic attacks only

Module (owner)	Level	GenAI produced	Team contribution	Why the design works	Known limits
Lifecycle and event store (S1)	L3	Transition table code, aggregate, PostgreSQL adapters, the 294-pair matrix test	State machine design (14 states), transition table, schema v2.3 with triggers and hash chain, ADRs 0001–0005	The database enforces the rules as well (foreign key to allowed transitions, exclusion constraint, append-only triggers), so an application bug cannot store an invalid state	–
Disputes and moderation (S3)	L2	Dispute and decision services, endpoints, moderator screens	Case workflow and decision rules from the use cases; upload deferral (D20, S3)	–	Audit screen and uploads in Iteration 2
Resident and moderator apps (S3)	L2	Expo screens, API client wiring, Jest tests	HCI analysis, wireframes W1–W7, both themes	–	Android and web only
CI/CD (S1)	L2	GitHub Actions workflow, Dockerfile, Render configuration	Choice of hosting and pipeline gates	–	No rollback rehearsal yet
Synthetic data (S3)	L2	Generator, validator and loader	Dataset requirements, scenario casts, the rule that public datasets supply shape statistics only	–	Synthetic, not real be-haviour

Module (owner)	Level	GenAI produced	Team contribution	Why the design works	Known limits
Design documents (all)	L2	Drafts of several Part A documents and two class diagrams; their headers say "drafted with Claude"	Every design decision (D1–D20, ADRs, the locality method choice), review and sign-off	–	–
This report (all)	L2	Draft text; an earlier draft was produced with GitHub Copilot	Structure, framing, and checking every figure against the code and test output	–	–

Errors found in AI output. Review, measurement and CI caught generated defects before merge, among them: the first version of S1, which flagged ordinary community members (DF-01); request bodies that silently ignored unknown fields (DF-03); a fuzz test that generated values JSON cannot encode (DF-06); a fuzz test that took 28 minutes (DF-07); an `assert` used as a runtime check (DF-08); and a stray control character written into a source file. These are reported to show that output was checked, not accepted as written.

What is not AI. The product contains no generative AI and makes no calls to an AI service. The assurance engine is a statistical model with deterministic, explainable rules. No real personal data was given to any tool; all data in the project is synthetic.

Accountability. Each member owns the modules in Table 3.3, has reviewed their design, code and tests, and can explain them. The team takes full responsibility for all submitted work.

8.8 External Components and Dependencies

Dependencies are pinned by lock files (`backend/uv.lock`, `frontend/package-lock.json`) and installed exactly as locked in CI. Table 8.3 is the dependency register (Form 13); all licences are open source and permit use in this project.

Table 8.3: Dependency register (versions as locked on 14 October 2026)

Component	Version	Licence	Use
Backend, runtime			
FastAPI	0.143.0	MIT	Web framework
Pydantic / pydantic-settings	2.14.0 / 2.15.0	MIT	Validation, settings
Uvicorn	0.54.0	BSD-3-Clause	ASGI server
SQLAlchemy	2.1.4	MIT	Database access
Alembic	1.20.0	MIT	Migrations
psycopg	3.3.6	LGPL-3.0	PostgreSQL driver (used unmodified as a library)
NumPy	2.5.4	BSD-3-Clause (with bundled MIT/0BSD/Zlib parts)	Numerics
SciPy	1.18.1	BSD-3-Clause	Beta quantiles
argon2-cffi	25.1.0	MIT	Password hashing
PyJWT	2.15.1	MIT	Tokens
python-multipart, PyYAML, structlog	0.0.32, 6.0.3, 26.1.0	Apache-2.0, MIT, MIT or Apache-2.0	Forms, config, logging
Backend, development only			
pytest, pytest-cov	9.1.1, 7.1.0	MIT	Tests, coverage
Hypothesis	6.169.0	MPL-2.0	Property tests, fuzzing
httpx, NetworkX	0.28.1, 3.7	BSD-3-Clause	API test client; k-hop cross-check
ruff, mypy, pre-commit	0.17.0, 2.4.0, 4.6.2	MIT	Lint, types, hooks
bandit, pip-audit	1.9.4, 2.10.1	Apache-2.0	Security scans
Locust	2.46.7	MIT	Load tests (suite 15)
Frontend, runtime			

Component	Version	Licence	Use
Expo SDK and Expo modules	57.0.x	MIT	App framework, router, secure store, image picker
React / React Native / react-native-web	19.2.3 / 0.86.3 / 0.21.4	MIT	UI
React Native Paper	5.15.3	MIT	Components
TanStack Query	5.104.1	MIT	Server state
openapi-fetch	0.17.0	MIT	Typed API client
react-hook-form, zod	7.90.0, 4.6.6	MIT	Forms, validation
Fraunces, Plus Jakarta Sans fonts	0.4.x	MIT and OFL-1.1	Typography
Frontend, development only			
TypeScript	6.0.3	Apache-2.0	Types
Jest, jest-expo, React Native Testing Library	30.5.2, 57.0.5, 14.1.0	MIT	Tests
ESLint, Prettier, openapi-typescript	9.39.5, 3.9.10, 7.13.0	MIT	Lint, format, client generation
Playwright	1.64.0	Apache-2.0	Browser tests (planned)
Services			
PostgreSQL	16	PostgreSQL Licence	Database (Neon in deployment)
GitHub Actions, Render, Neon	–	Free tiers	CI, hosting, database

8.9 Risks and Mitigation

Table 8.4: Risk register

Risk	Level	Mitigation
Parameters (thresholds, half-life, weights) are wrong for real neighbourhoods	High	Kept in versioned configuration with sensitivity ranges; FYP-2 sensitivity study and time-ordered hold-out; results reported with `config_version`
False suspicion of legitimate dense groups	High	Corroboration rule (ADR-0007), C3 guard test, neutral wording, human decision before any restriction
Synthetic data differs from real behaviour	Medium	Shape statistics from public trust networks; separate pilot data in FYP-2, never mixed with simulation results
Members cannot explain AI-assisted code at the viva	Medium	Line-by-line review, plain-language notes in the code, each member presents their own modules in the demo
Free hosting sleeps or changes terms	Medium	Local run as a demo fallback; deployment configuration kept in the repository
Location privacy breach	High	No location in requests, responses or logs; tested on every build (suite 14)
Scope creep into stretch modules	Medium	Frozen scope; changes only through a change request (Form 12)
Flaky frontend tests on slow machines	Low	Raised first-render timeout; tests run alone in CI (DF-09)

8.10 Security and Privacy Considerations

- Passwords are stored as argon2 hashes; tokens are JWTs with a secret held in environment settings, never in the repository; HTTPS is terminated by Render.
- Every transition checks the actor's role; refused commands are recorded in a hash-chained audit log; moderators act only in their own community.
- Request bodies refuse unknown fields and NUL characters; fuzzing checks that bad input gives 422, never a server error or database message.
- No exact location, address or CNIC is stored or returned; the privacy suite scans every response model, a whole loan's API traffic, log lines and audit rows.

- Synthetic data only; every synthetic file and seed account is labelled synthetic.
- bandit, pip-audit, npm audit and Dependabot run on the dependencies.

8.11 Current Limitations

- The engine's parameters are unvalidated hypotheses; no claim is made yet about decision quality against baselines.
- The locality method, uploads, scheduler jobs, reviews, three manipulation signals, information requests, appeals, notifications and the audit-log screen are not built.
- Two of the thirteen scenarios (ES-09, ES-10) cannot run until the locality method exists.
- Transition latency under load and the deployed-instance system tests have not been run.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

The committed core is partly built (Chapter 6). What remains is to complete the core and enabling modules (1, 3, 5, 7, 9 and 10), to add the supporting search module, and to evaluate the engine. Stretch modules (events, forum and chatbot, vendors) are considered only if the core is integrated and the evaluation is on track, at most one in Iteration 3 and none in Iteration 4.

9.2 Remaining Modules and Features

Table 9.1: Module roadmap, Iterations 2 to 4

Module	Core part finished	Whole module	Evaluated
1 User & Community	It. 2: membership, locality (A3, A4)	It. 3: accounts, profile, blocking	It. 4: locality test cases, pilot privacy
3 Resource Sharing	It. 2: scheduler, no-show, extensions, overdue, counter-offer	It. 3: listings, calendar	It. 4: load and regression

Module	Core part finished	Whole module	Evaluated
4 Search	–	It. 3	It. 4: precision@k
5 Trust	It. 2: all roles and metrics, reviews, remaining signals, ES-07 to ES-12	It. 3: trust profile, reporting	It. 3 preliminary, It. 4 full
7 Communication	It. 2: in-app notifications	It. 3: chat, e-mail, push	It. 4
9 Administration	It. 2: information requests, appeal, reversal, audit screen	It. 3: user, community and report management	It. 4: pilot moderators
10 Analytics	It. 2: simulator, baselines, evaluation plan	It. 3: held-out attack generator	Runs the evaluation in It. 4

9.3 Remaining Iterations

- **Iteration 2 (FYP-1 final):** the Iteration 2 plan in Section 3.5; target about 45% of committed scope, all thirteen scenarios in CI, the first baseline comparison and the written FYP-2 evaluation plan.
- **Iteration 3 (FYP-2 mid):** all major modules connected; search; the held-out attack generator, written by S3 so that it is independent of the detector's author (S2); a preliminary comparison with the Iteration 2 baseline.
- **Iteration 4 (FYP-2 final):** the full evaluation, the pilot, no new scope.

9.4 Implementation Plan

The method that worked in Iteration 1 continues: design first (each new use case dressed, each algorithm worked by hand), then vertical slices with a release pull request each, every

change behind CI. Event types or schema fields needed by new features (for example reviews and notifications) go through the frozen-schema sign-off by all three members before code is written.

9.5 Testing and Validation Plan

The FYP-2 evaluation asks whether the engine's uncertainty and safeguards actually improve decisions.

- **Baselines:** global average, role-specific average, recency-weighted average, Beta posterior without indirect evidence, direct evidence only, direct evidence with uncertainty, direct with k-hop, and the full system.
- **Ablations:** without uncertainty, without indirect trust, without manipulation detection, without the context policy. A component with no measurable benefit is reported as such.
- **Metrics:** Brier score, log loss, interval coverage and calibration; false-positive and false-negative rates of the manipulation signals; escalation precision and recall; coverage and latency at $k = 1$ to 4; fairness between new and established users.
- **Data:** a time-ordered hold-out on simulated neighbourhoods; public trust networks (Epinions, Bitcoin OTC) for propagation only; a pilot with real residents for acceptance (task completion, SUS, understanding of the explanation), reported separately and collected with consent. The three sources are never mixed.
- **Targets:** the proposal stated $\geq$70% accuracy and $<$5% false positives; FR/NFR v1.3 leaves the numeric targets for decision quality open until a baseline exists. The targets will be fixed in the written evaluation plan at the end of Iteration 2, before any evaluation run.

9.6 Deployment Plan

Render (API and web) with Neon (PostgreSQL) remains the target. The deployment diagram has been redrawn for this target (Figure 5.9). Before FYP-1 final: rehearse a rollback to the previous deploy, add the post-deploy smoke test (TC-91), and produce an Android build for device testing (TC-73). For the pilot: a paid tier or a small dedicated host if free-tier sleep affects users, decided with the supervisor.

9.7 Tentative Timeline

Table 9.2: Timeline

Iteration	Period	Focus
1	Aug–Oct 2026	Design and analysis; live core (done)
2	Nov 2026–Jan 2027	Core modules completed; simulator, baselines, evaluation plan
3	Feb–Apr 2027	Integration, search, attack generator, preliminary evaluation
4	May–Jun 2027	Evaluation, pilot, final report

9.8 Expected Final Deliverables

A deployed platform with the committed core; the assurance engine with its evaluation against baselines and ablations; the scenario suite and simulator in CI; the pilot results; the design baseline kept in step with the code; the final report, poster and demo; and the AI usage log and dependency register kept current.

Chapter 10

Conclusion

Digital Mohalla addresses one question: whether a proposed neighbourhood loan has enough trustworthy evidence, when that evidence is sparse, role-dependent, indirect and open to manipulation. Research on reputation systems showed that averages hide uncertainty and that newcomers need a policy, not a score; research on collusion showed that a signal must separate rings from ordinary two-sided reviewing. These findings became the requirements and the design: four evidence states instead of a trust number, a recency-weighted Beta posterior with an interval, capped indirect evidence, corroborated manipulation signals, and a human moderator for every hold and dispute.

Iteration 1 built that core and the lifecycle that feeds it. The lifecycle is event-sourced, replayable and protected by the database against invalid transitions and double bookings; the engine runs on real outcomes and explains every result; disputes and holds reach a moderator whose decision becomes evidence. 623 backend and 29 frontend tests pass, all seven scenarios required at mid-term are correct in state and explanation, and the system is deployed from `main` after every successful CI run. Generative AI assistance is declared per module, with the team's own design decisions and reviews recorded alongside it. The main risk is that the engine's parameters are still hypotheses. Iteration 2 completes the core modules and builds the simulator and baselines, so that FYP-2 can test, with numbers, whether the engine's uncertainty leads to better lending decisions than the averages it replaces.

Appendix A

Proposal Action Register

The register itself is Table 3.1 in Section 3.2. This appendix gives the supporting evidence for the location action, the panel point that required evidence beyond the implementation.

A.1 Why location matters: threat model

Every loan in Digital Mohalla ends in a physical handover at or near a member's home, so membership of the community is itself a safety claim. The A11 design and its test cases cover the following risks; without a locality check:

- **Outsiders (visitors).** A person who does not live in the neighbourhood joins, borrows and is never seen again; nobody nearby knows them, so there is no social or community evidence about them.
- **GPS spoofing.** Mock-location apps let anyone report a position inside the community.
- **Duplicate accounts (Sybil).** One person with several accounts can build fake history or return after a dispute under a new name.
- **Genuine residents wrongly rejected.** Homes on the boundary, poor indoor GPS, refusal of location permission and newcomers who know nobody must not be shut out.

The method therefore has to resist spoofing and outsiders while never rejecting a resident automatically, and it must do so without storing anyone's exact location.

A.2 Locality method comparison (design step A11)

Five candidate methods were rated before any measurement (design judgements; NFR-Q-04 measures the chosen method later).

Table A.1: Candidate locality methods

Method	False accepts	False rejects	Privacy	Spoofing resistance	Cost
Coarse GPS geofence alone	Medium	Medium	Medium	Low	Low
Community invitation alone	Medium	Medium–high	High	Medium	Low
Moderator approval alone	Low	Low	Medium	High	High
Address evidence, shown not kept	Low–medium	Medium	Low	Medium	Medium–high
Geofence + invitation, moderator for the rest (chosen)	Low	Low	Medium–high	Medium–high	Medium

Why the combination: neither signal alone is enough. GPS is easy to spoof, and invitations alone ignore the location question the panel raised. Together, a spoofer must also obtain a real member's single-use invitation. Uncertainty costs moderator time, never a rejection (FR-A-08).

A.3 Locality test cases

The reference rule (`docs/design/12_locality/locality_rule.py`) was checked on synthetic inputs: 12 checks in the six test-case groups of the project overview, plus 4 invitation-error checks and 2 privacy and no-rejection checks; all 18 pass.

A.4 Privacy boundary in the built system

A borrowing request carries no location (decision D13); the lifecycle reads the member's stored coarse result. Tests TC-58 and TC-60 check that no response model contains a

Table A.2: Locality test cases (SYNTHETIC inputs)

ID	Case	Input	Expected = actual
L1a–c	GPS spoofing	Mock-location provider; undetected spoof without invitation; spoofed fix 2 km away	CONFLICTING; INCONCLUSIVE; CONFLICTING
L2a–c	Boundary users	30 m inside or outside the edge at 20 m accuracy; 150 m inside	INCONCLUSIVE; INCONCLUSIVE; VERIFIED
L3	Visitors	Inside, no invitation	INCONCLUSIVE
L4	Permission refused	No location, valid invitation	INCONCLUSIVE
L5	Duplicate accounts	Inside, invitation, duplicate signal	CONFLICTING
L6a–c	Genuine residents	At home; indoors with 250 m accuracy; new resident without invitation	VERIFIED; INCONCLUSIVE; INCONCLUSIVE
E1–E4	Invitation errors	Expired, used, wrong community, self-invitation	Error, retry
P1	Privacy	Fields stored after a check	No coordinates or accuracy
P2	No automatic rejection	64 combinations of inputs	Only VERIFIED, INCONCLUSIVE or CONFLICTING

location or identity field, that a whole loan through the API reveals no personal data, and that a request body carrying a latitude or an address is refused with 422.

Appendix B

Detailed Requirements

The full texts are `DigitalMohalla_Use_Cases_Ver1.md` and `DigitalMohalla_FR_NFR_Ver1.md` (baseline v1.3) in `docs/design/01_requirements/`, with the traceability matrix in `Traceability_Matrix.md`.

B.1 Dressed core use cases (summary)

Table B.1: Main success scenarios and key extensions of the core use cases

Use case	Summary
B6 Respond	The lender sees the request with its evidence summary and accepts or declines. Acceptance commits under the overlap constraint, blocks the dates and declines overlapping pending requests; a refused acceptance ends DECLINED "dates taken". The borrower may cancel from any pending state.
B7 Handover	The lender records the condition checklist (and deposit, record only); the borrower confirms receipt; the loan becomes ACTIVE only when both have confirmed, in that order.
B8 Return	The borrower initiates the return; both confirm condition; the loan passes RETURNED to COMPLETED through `transaction_closed`, on time or late; outcomes go to the engine and the booking is released.

Use case	Summary
B12 Dispute	Either party, while ACCEPTED, ACTIVE or RETURN_ INITIATED, raises a dispute with one of five reasons; the loan is DISPUTED with its previous state stored, frozen, and a case opens. Only the raising party may withdraw, which restores the previous state. A second dispute shows the open case.
D3/4/5 Assess	For every request, the engine returns one of four states with interval, evidence, signals, safeguards and explanation; failure or timeout holds the request (`engine_failure`).
D6 Escalate	A dispute or hold opens a case with trigger, priority, reason and evidence trail, attached to an open case if one exists.
I4 Resolve	A moderator of the community decides with a mandatory reason: a dispute closes through `dispute_resolved` with a finding; a hold is cleared, cleared with safeguards, or restricted (declined); no safeguards on an engine-failure hold.
I6 Audit	Every transition is in the hash-chained event log and refused commands in the audit log; the viewing screen comes in Iteration 2.
A3/A4 Membership	A member selects a community, passes the A11 locality check (one in-memory GPS reading and an invitation), and a moderator approves or decides uncertain cases (Iteration 2).

B.2 Requirement counts

68 use case IDs; functional requirements in eight groups (A accounts, B borrowing, S search, T trust, M moderation, C communication, N analytics, and condensed stretch groups); 35 functional requirements are CORE-tagged in Modules 1, 3, 5, 9 and 10, and 25 of them have a Given/When/Then acceptance criterion in Part E. Non-functional requirements are grouped as reliability (R), performance (P), decision quality (Q), security (S), privacy (PR), usability and explainability (U) and maintainability and deployment (D); numeric gates are set only where Iteration 1 can measure them.

Appendix C

Detailed Test Cases

Test case texts (input, expected result, traceability) are in Test Plan v1.3 §6. Table C.1 lists the test cases that are automated and passing on 14 October 2026, with the test file that implements each.

Table C.1: Automated test cases by suite

Test cases	Suite	Test files
TC-01, 02, 05	1 Transition matrix	`test_transition_table`, `test_borrowing_service`
TC-03, 06	1 Hold exits; stale command	`test_moderation`, `test_event_store`
TC-07	2 Replay	`test_borrowing_service`, `test_event_store`, `test_aggregate`
TC-08, 09, 15	3 Double booking, soft hold	`test_borrowing_service`
TC-10, 11, 12	4 Authorisation	`test_borrowing_service`, `test_moderation`, `test_api`
TC-13, 14	5 Append-only, tamper	`test_event_store`
TC-20 to 24	6 Beta posterior	`test_beta_posterior`, `test_posterior_properties`, `test_outcome_extraction`
TC-25 to 28	7 k-hop	`test_khop`, `test_trust_edges`, `test_khop_on_dataset`
–	8 k sweep (reported)	`scripts/khop_sweep.py`
TC-29 to 35, 39	9 Decision policy	`test_decision_policy`, `test_assessment_rules`

Test cases	Suite	Test files
TC-36, 37	10 Engine failure	`test_borrowing_service`
TC-40 to 47	11 Scenarios	`test_fixture_scenarios`, `test_dataset_scenarios`
TC-30, 38, 43, 44	12 Dense group, manipulation	`test_manipulation`, `test_assurance_engine`
TC-50 to 57	13 Disputes and holds	`test_disputes`, `test_api_disputes`, `test_moderation`, `test_moderation_rules`, `test_api_moderation`
TC-58, 60	14 Privacy	`test_privacy`, `test_api`
TC-64, 66, 67, 68	16 Handover, return, outcome feed	`test_borrowing_service`, `test_outcome_feed`, `test_outcome_rules`
TC-71, 74 to 77	Unit and contract	`test_assurance_engine`, `test_explanation`, `test_outcome_rules`, `test_similarity`, `test_engine_config`
TC-80, 81, 84	17 API security	`test_api`, `test_privacy`, `test_fuzz`
TC-100, 105 to 107	19 Screens W1, W5, W6, W7	`lender-evidence-test`, `moderation-screens-test`, `moderation-lib-test`, `test_lender_evidence_api`

Not yet automated. TC-16 (replay property test), TC-61 (transition latency under load), TC-63 and TC-90 to TC-92 (deployed-instance demo and smoke tests), TC-70 (duplicate request), TC-73 (Android device), TC-82, TC-83, TC-86 to TC-88 (further API and host security checks), TC-101 to TC-104 (screens W2 to W4). **Deferred to Iteration 2:** TC-59 and TC-85 (uploads, decision D20).

Appendix D

Repository and Contribution Evidence

- Repository: `https://github.com/Areeba444/DigitalMohalla`
- Live API: `https://digitalmohalla-api.onrender.com/health`
- Live web app: `https://digitalmohalla-web.onrender.com`

Table D.1: Merged pull requests (commit owners from the pull request's commits)

PR	Title	Commit owner(s)	Reviewer
1	Design baseline	All three	Fatima
2	Borrowing lifecycle core (event store, transition table, migration)	Fatima	Areeba
3	Versioned engine configuration and Slice A stub engine	Areeba	Minahil
4	Minimal login and REST API	Fatima	Areeba
5	Resident web app for the Slice A loop	Minahil	Fatima
6	Synthetic test dataset, validator and loader	Minahil	Areeba
7	GitHub Actions checks	Fatima, Areeba	Areeba
8	Release: develop to main	–	–
15	Slice B kickoff (test plan v1.3, branch plan, D10–D16)	All three	Minahil
16	Outcome extraction (B1)	Areeba	Fatima
17	Deploy the API to Render with Neon	Fatima	Areeba

PR	Title	Commit owner(s)	Reviewer
18	Release: outcome extraction, Render deployment	–	–
19	Recency-weighted Beta posterior and similarity (B2)	Areeba	Fatima
20	Deploy the web app to Render	Fatima	Areeba
21	Trust-edge projection from verified outcomes (B2)	Fatima	Areeba (after merge, 14 Oct)
22	Bounded k-hop indirect trust and k sweep (B3)	Fatima	Areeba
23	Decision policy, explanation and real engine (B4)	Areeba	Fatima
24	Release: assurance engine, web deployment	–	–
25	Scenario suite (B5)	Minahil	Areeba
26	FYP defence slide document	Minahil	Fatima
27	Modern mobile design, two themes	Fatima	Areeba (after merge, 14 Oct)
28	Lender evidence screen (B6)	Minahil	Fatima
29	Release: Slice B complete	–	–
30	Release: mobile UI	–	–
31	Manipulation-risk checks with retaliation handling (C1)	Areeba	Fatima
32	Raise and withdraw disputes (C1)	Minahil	Areeba
33	Moderator decisions on disputes and holds (C2)	Minahil	Areeba
34	Privacy checks, strict request bodies, fuzzing (C2)	Fatima	Areeba
35	Moderator app and dispute screens (C3)	Minahil	Areeba
36	Release: Slice C, feature freeze	–	–

Pull requests 9 to 14 are open Dependabot updates. Where a branch also contains a small number of commits by another member (for example a shared fix), only the main owner

is listed.

Appendix E

Additional Design Diagrams

E.1 Class diagrams

Each member drew the classes of their own packages (design step A7); every class names the requirements it realises. Table 5.1 maps these classes to the code.

Reasoning. The lifecycle aggregate depends only on the transition table and the ports, so it can be tested without a database; the PostgreSQL event store implements the port. Indirect trust exposes one operation to Student 2's engine. *Serves:* FR-B-05, B-08, B-09, FR-T-04.

Reasoning. A single facade runs the pipeline and returns one result object in which trust and manipulation risk are separate attributes, never averaged. Signal checks share one interface so that the Iteration 2 signals (S2, S3, S5) are added without changing the detector. The class named `ReciprocalClustering` realises S1, which decision D18 redefined as partner clustering. *Serves:* FR-T-01 to T-08.

Reasoning. A moderator decision is a value that the case service turns into exactly one lifecycle command, so moderation cannot bypass the transition rules. *Serves:* FR-T-10, FR-T-11, FR-M-05.

E.2 Sequence (b): handover and return to evidence

Reasoning. An outcome is written only after both parties have confirmed, which is what makes it "verified" evidence (FR-T-03); the close hook writes the outcomes and refreshes the affected trust edges in the same database transaction as `transaction_closed`. *Owner:* S1.

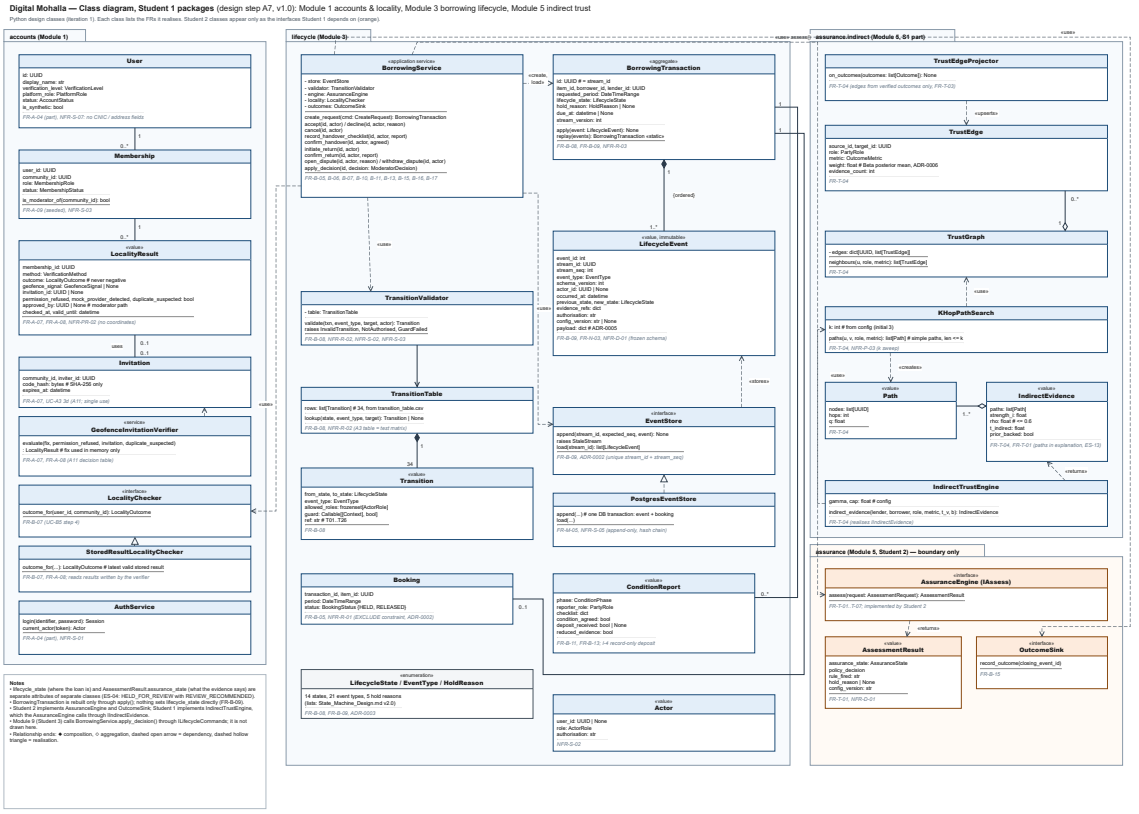


Figure E.1: Class diagram, Student 1 packages: accounts and locality, lifecycle and event store, indirect trust

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/05_class_model/class_diagram_s1.drawio`

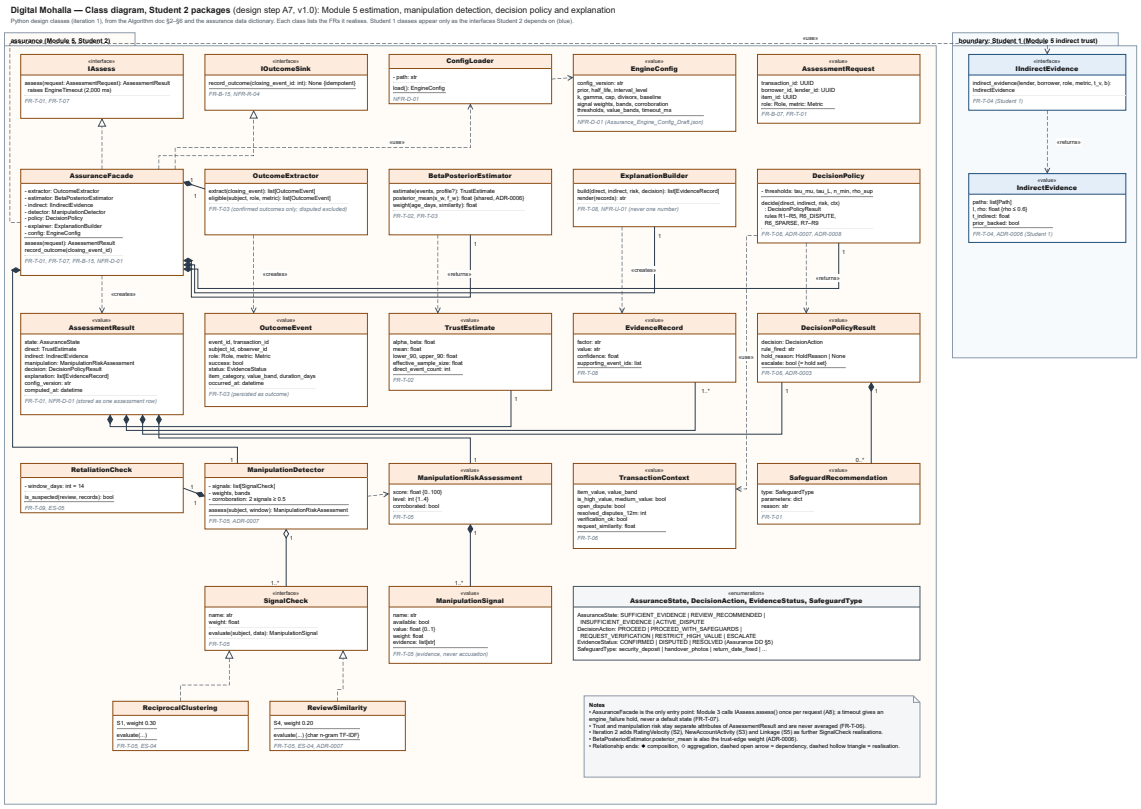


Figure E.2: Class diagram, Student 2 packages: estimation, manipulation detection, decision policy and explanation

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/05_class_model/class_diagram_s2.drawio`

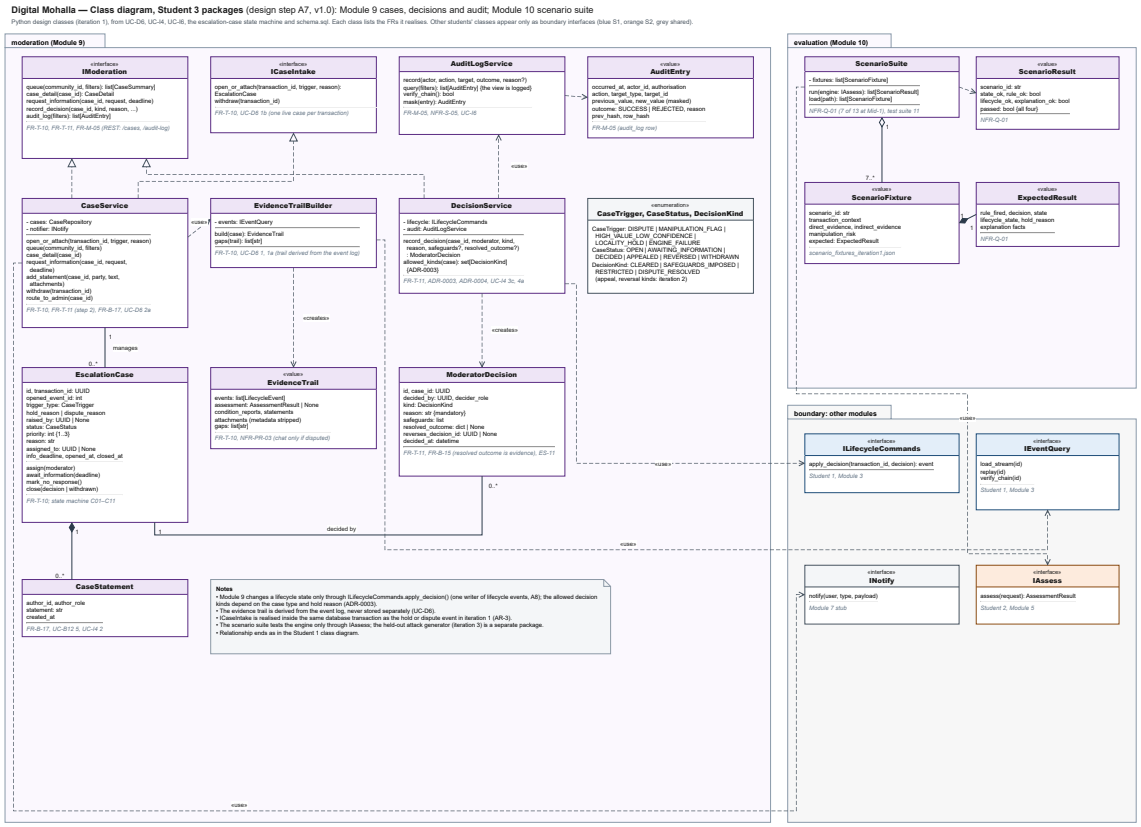


Figure E.3: Class diagram, Student 3 packages: cases, decisions and audit (Module 9) and the scenario suite (Module 10)

Full-size diagram: Open the full-size diagram in draw.io
Source: `docs/design/05_class_model/class_diagram_s3.drawio`

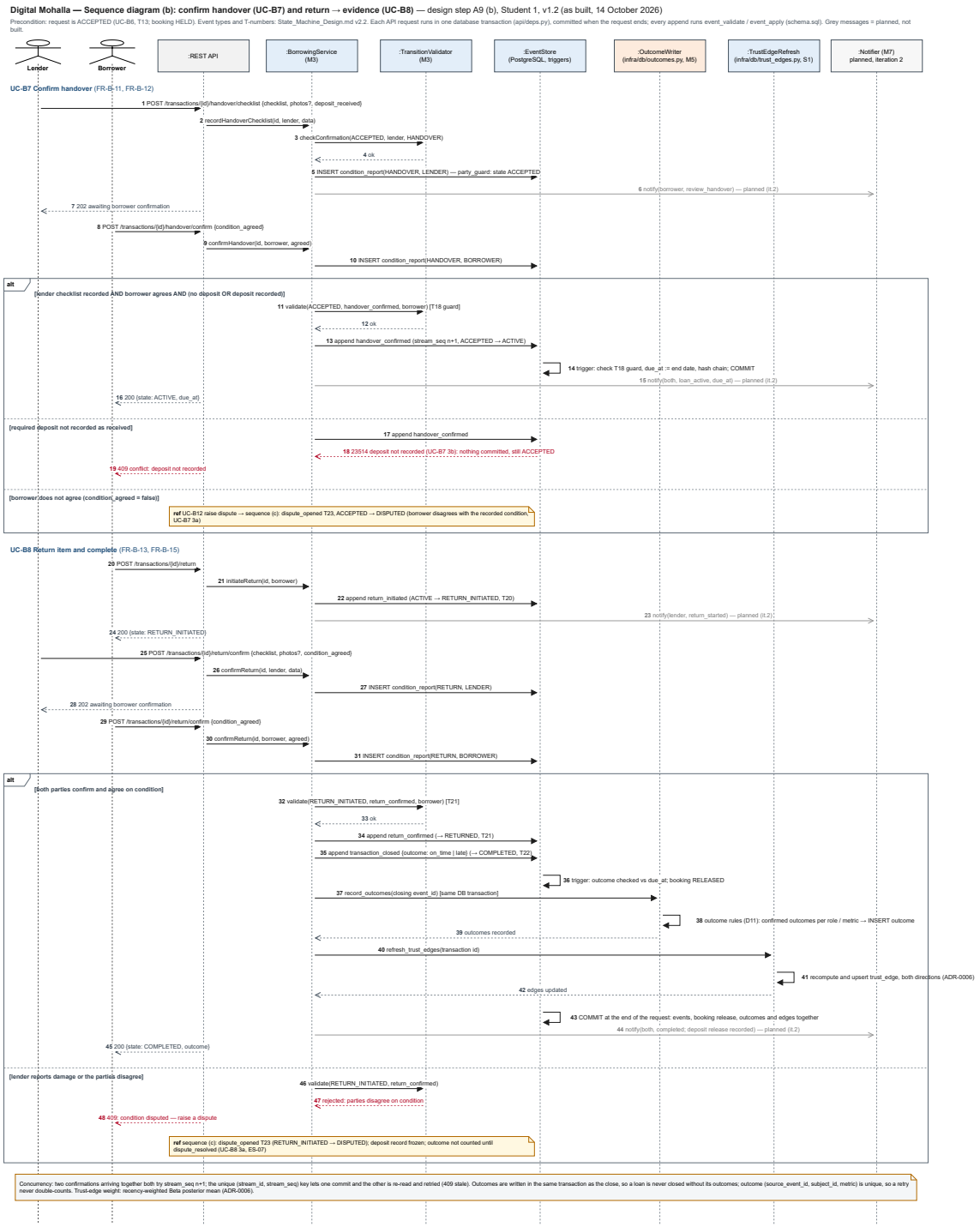


Figure E.4: Sequence (b): dual confirmation of handover and return, then outcome to the engine (UC-B7, B8)

Full-size diagram: Open the full-size diagram in draw.io

Source: `docs/design/08_sequence_diagrams/sequence_b_handover_return.drawio`

E.3 Deployment as designed in Part A (historical)

The provisional Part A deployment design (design step A13) is kept for reference. It placed the whole system on one AWS EC2 host with Docker Compose and images in GitHub Container Registry, and marked the host “to be confirmed”. The built system runs on Render and Neon instead (Section 5.8.5, Figure 5.9).

E.4 Wireframes

The wireframes (design step A12, S3) were drawn from the HCI analysis; every screen that shows an assurance state shows its evidence (NFR-U-01). The built screens follow them.

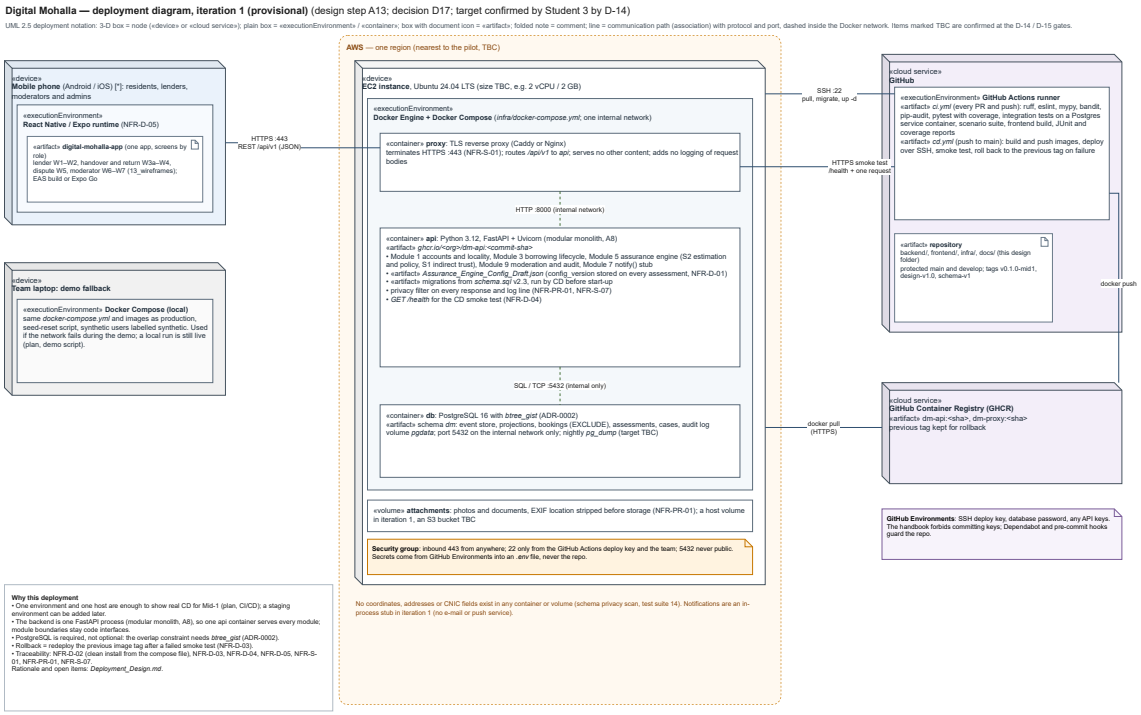


Figure E.5: Deployment diagram as designed in Part A (A13, provisional; superseded by the as-built deployment)

Full-size diagram: Open the full-size diagram in draw.io

Source: `docs/design/04_deployment/archive_v1/deployment_diagram_v0.9_partA_provisional.drawio`

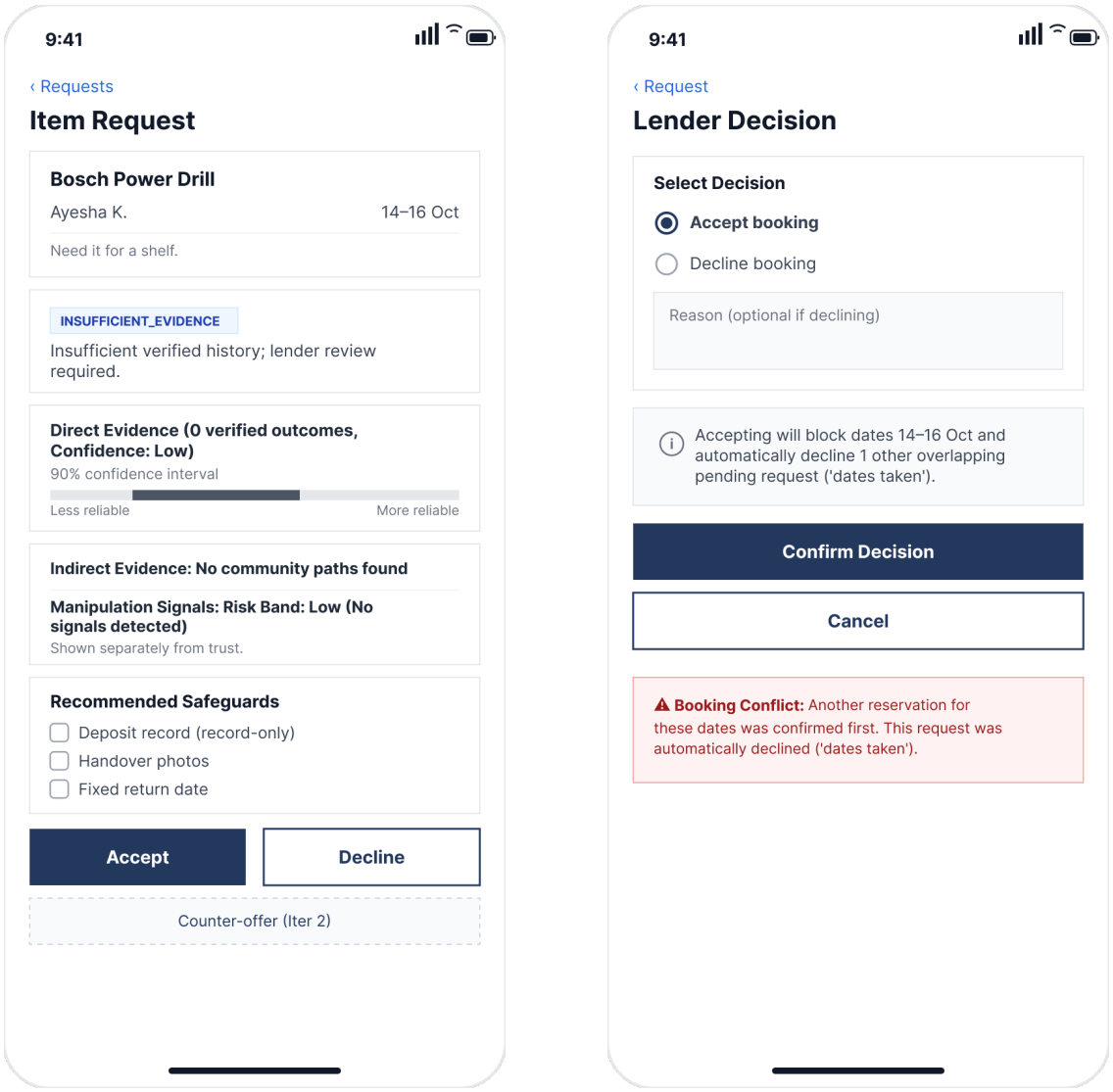


Figure E.6: W1 lender request with assurance summary; W2 lender decision and date conflict

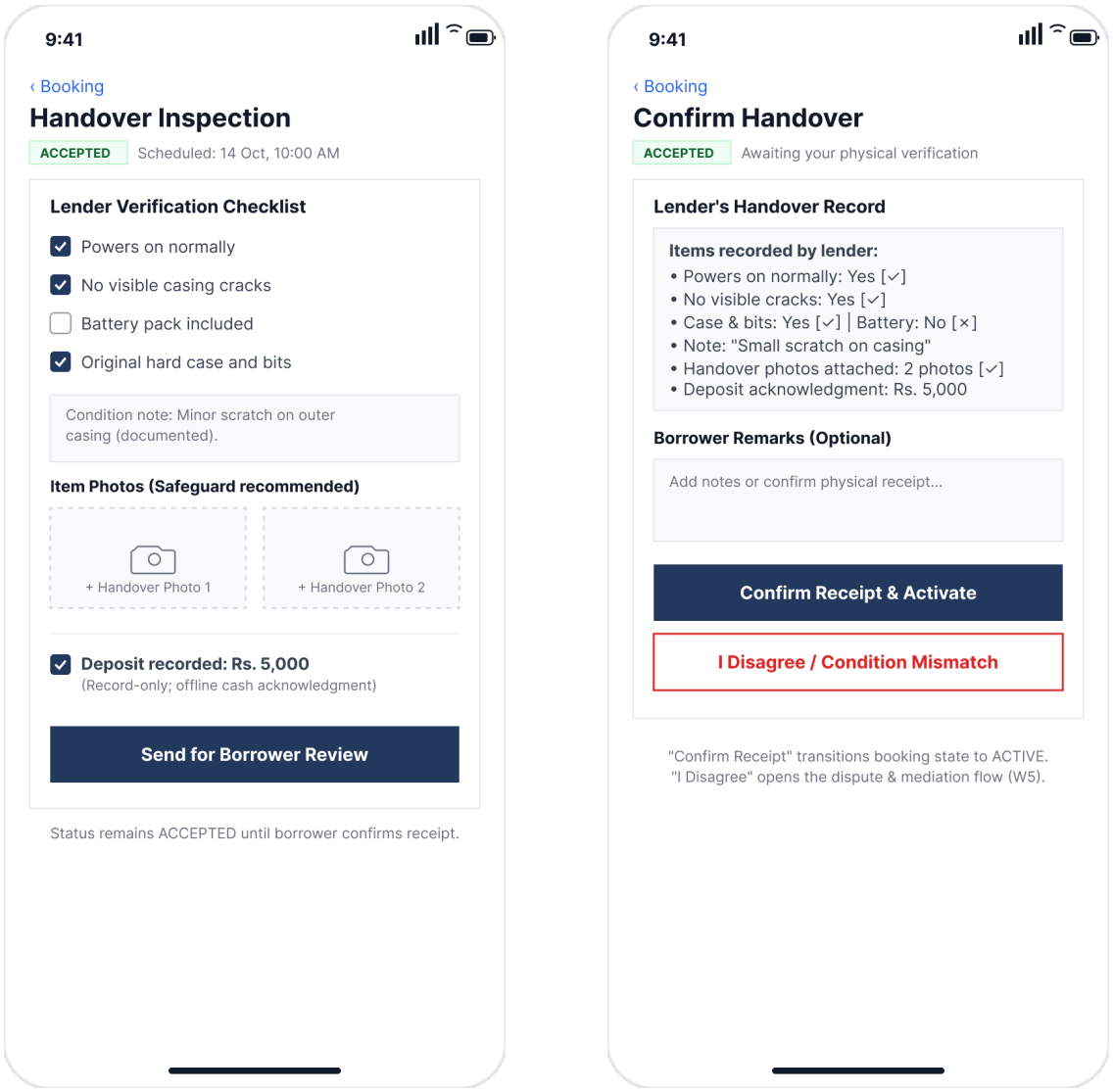


Figure E.7: W3a handover inspection (lender); W3b handover confirmation (borrower)

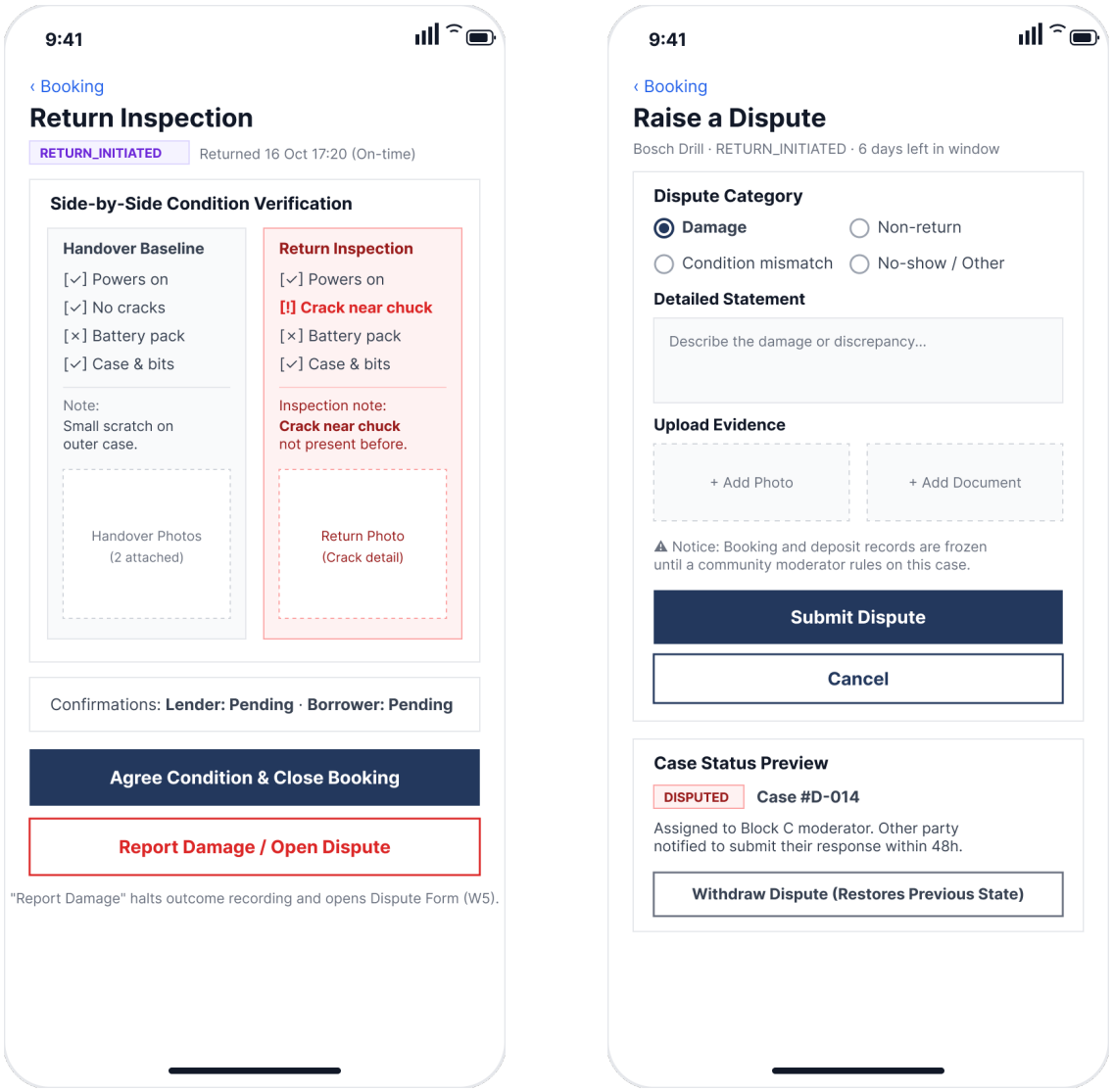


Figure E.8: W4 return inspection; W5 dispute form and progress

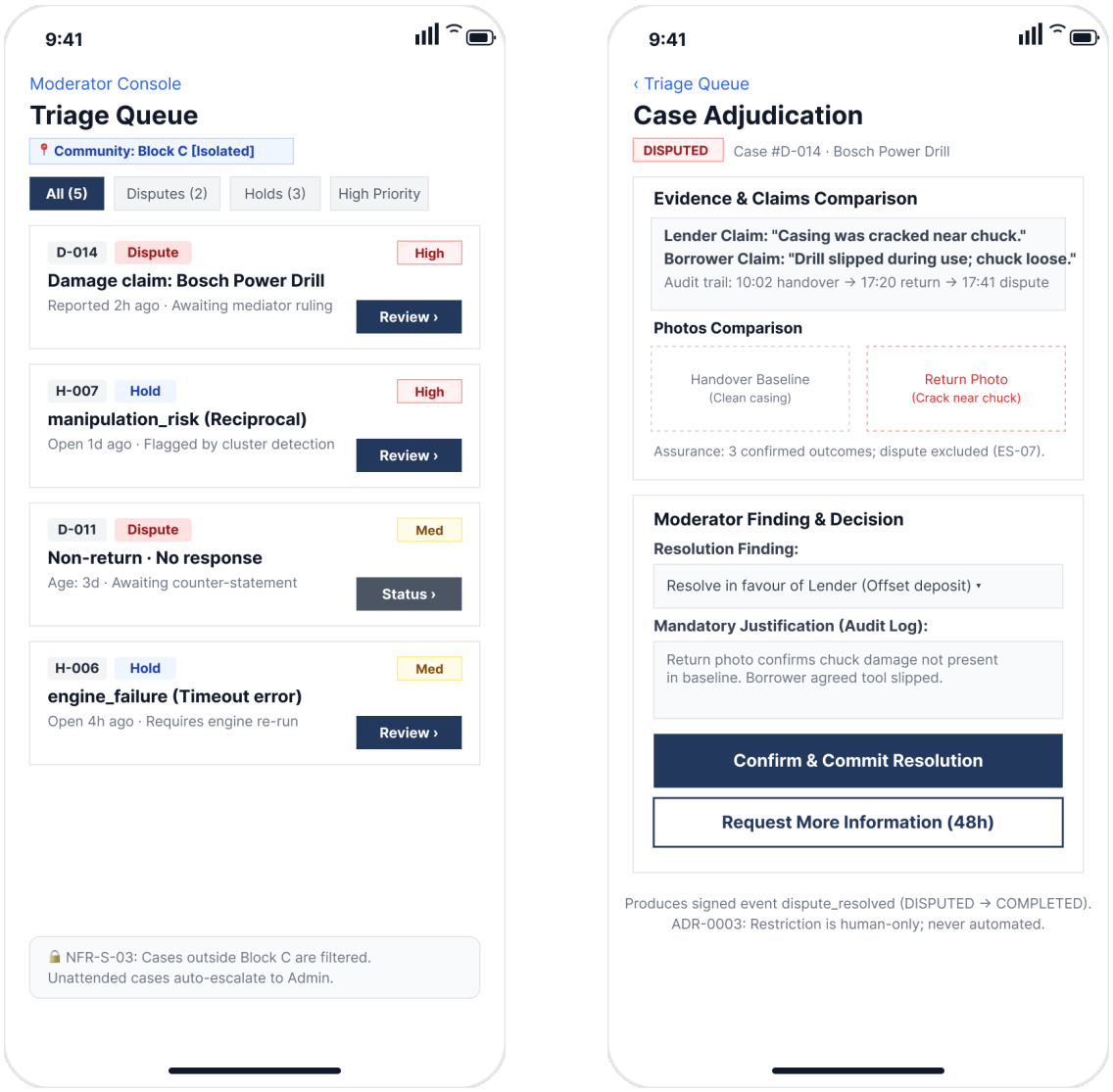


Figure E.9: W6 moderator queue; W7 case adjudication

Appendix F

Meeting and Progress Evidence

Team meetings that produced decisions are recorded in `docs/design/00_management/`.

Table F.1: Recorded team meetings and reviews

Date	Record	Outcome
11 Oct 2026	Team decisions (design steps A1, A4)	D1–D9: `payload` field (ADR-0005); internal event types named; ADR-0001 to 0004 accepted; no disputes from RETURNED; "RESUME" dropped; booking hold wording; Request table merged; cancellation from any pending state; case state WITHDRAWN
11 Oct 2026	Integration decisions	The three members' Part A artefacts made one baseline: ADR-0007 and ADR-0008, one vocabulary, engine configuration 1.1.0, privacy boundary at membership, provisional deployment, folder structure
11 Oct 2026	Design review A14	Every diagram checked against its notation standard; seven faults found and fixed; class diagrams S2 and S3 added

Date	Record	Outcome
13 Oct 2026	Team decisions (Slice B kickoff)	D10–D16 at kickoff; D17 decided by S2 during B2: edge weight, outcome mapping, thresholds, ES-05 and ES-13 expectations, device scope, test plan use, similarity
13–14 Oct 2026	Team decisions (Slice C)	D18–D20: partner clustering, retaliation handling, upload deferral; confirmed 14 October

Bibliography

[1] E. Ert, A. Fleischer, and N. Magen, "Trust and reputation in the sharing economy: The role of personal photos in Airbnb," *Tourism Management*, vol. 55, pp. 62–73, 2016.

[2] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[3] A. Jøsang and R. Ismail, "The Beta reputation system," in *Proceedings of the 15th Bled Electronic Commerce Conference*, Bled, Slovenia, 2002.

[4] A. Jøsang, *Subjective Logic: A Formalism for Reasoning Under Uncertainty*. Cham: Springer, 2016.

[5] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, "Reputation systems," *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.

[6] J. A. Golbeck, "Computing and applying trust in web-based social networks," Ph.D. dissertation, University of Maryland, College Park, 2005.

[7] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The EigenTrust algorithm for reputation management in P2P networks," in *Proceedings of the 12th International Conference on World Wide Web*, 2003, pp. 640–651.

[8] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: a survey," *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626–688, 2015.

[9] A. Mukherjee, B. Liu, and N. Glance, "Spotting fake reviewer groups in consumer reviews," in *Proceedings of the 21st International Conference on World Wide Web*, 2012, pp. 191–200.

[10] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 895–904.

[11] G. Bolton, B. Greiner, and A. Ockenfels, "Engineering trust: reciprocity in the production of reputation information," *Management Science*, vol. 59, no. 2, pp. 265–285, 2013.

[12] A. Fradkin, E. Grewal, and D. Holtz, "Reciprocity and unveiling in two-sided reputation systems: Evidence from an experiment on Airbnb," *Marketing Science*, vol. 40, no. 6, pp. 1013–1029, 2021.

[13] P. Massa and P. Avesani, "Trust-aware recommender systems," in *Proceedings of the 2007 ACM Conference on Recommender Systems*, 2007, pp. 17–24.

[14] S. Kumar, F. Spezzano, V. S. Subrahmanian, and C. Faloutsos, "Edge weight prediction in weighted signed networks," in *Proceedings of the IEEE 16th International Conference on Data Mining (ICDM)*, 2016, pp. 221–230.

[15] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, and V. S. Subrahmanian, "REV2: Fraudulent user prediction in rating platforms," in *Proceedings of the 11th ACM International Conference on Web Search and Data Mining*, 2018, pp. 333–341.

[16] OLX Pakistan, "OLX pakistan classifieds," https://www.olx.com.pk.

[17] Meta Platforms, "Facebook marketplace," https://www.facebook.com/marketplace.

[18] H. Mehmood, T. Ahmad, L. Razaq, S. Mare, M. Z. Usmani, R. J. Anderson, and A. A. Raza, "Towards digitization of collaborative savings among low-income groups," *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 35:1–35:30, 2019.

[19] Nextdoor Holdings, "Nextdoor," https://nextdoor.com.

[20] Peerby, "Peerby: borrow and rent from people nearby," https://www.peerby.com.

[21] Fat Llama, "Fat llama: peer-to-peer rental," https://fatllama.com.

[22] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2002, pp. 253–260.

[23] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020.

[24] M. Fowler, "Event sourcing," https://martinfowler.com/eaaDev/EventSourcing.html, 2005.

[25] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[26] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation: Exclusion constraints and the btree_gist module," https://www.postgresql.org/docs/16/, 2023.